

Deep Learning — Exam Cheat Sheet

IST · Open-book final · Part 1 (8 MCQ · 32) + Part 2 (3 short · 18) + Part 3 (2 problems · 50) · 120 min

Part 3 is fixed: **Problem 1 = CNN** · **Problem 2 = sequence/attention**. Those two = 50 pts. The recipe boxes below win the exam.

★ Exam Recipes (highest yield)

CNN forward pass (conv → ReLU → pool)

Input X , kernel K ($k \times k$), stride S , pad P , bias b :

1. Pad X with P zero-rings.
2. Each output cell = (element-wise $K \odot \text{patch}$) + b . No kernel flip (it's cross-correlation).
3. Out size $M' = \lfloor \frac{M-k+2P}{S} \rfloor + 1$.
4. ReLU $\max(0, \cdot)$ elementwise.
5. Max-pool $p \times p$, stride s : max per window; out $\lfloor \frac{M'-p}{s} \rfloor + 1$. Avg-pool = mean.

CNN parameter count — whole net (step by step)

Track the shape $H \times W \times C$ layer by layer, then sum params:

1. Conv/pool spatial size: $M' = \lfloor (M - k + 2P) / S \rfloor + 1$. (pad $P=\{ \}$, $k=\{ \}$, $S=\{ \}$ $\Rightarrow$ same size; pool 2×2 stride 2 $\Rightarrow$ halves H, W .)
2. Channels C = the number of filters of the last conv (pool keeps channels).
3. Conv params: $(k \cdot k \cdot C_{\text{in}} + 1) \cdot C_{\text{out}}$ — +1 = bias/filter; C_{in} = input channels (not spatial size!). Pool & activations: **0** params.
4. Flatten = $H_{\text{last}} \cdot W_{\text{last}} \cdot C_{\text{last}}$ (product of the last feature-map dims).
5. FC (linear) $n_{\text{in}} \rightarrow n_{\text{out}}$: $(n_{\text{in}} + 1) \cdot n_{\text{out}}$ — +1 = bias/output neuron. Total = sum of all conv + FC params. Conv params are independent of input size; the FC count needs the flatten (so track shapes!).
 $1 \times \{ \} 1$ conv = channel mixing; factorized $(k \times \{ \} 1)$ then $1 \times \{ \} k$ = fewer params, same field.

RNN forward pass (Prob 2, 2021–23)

$$h_t = g(W_{hx}x_t + W_{hh}h_{t-1} + b_h)$$

$$\text{logits } o_t = W_{yh}h_t + b_y$$

$h_0 = \mathbf{0}$, usually $g = \text{ReLU}$. Compute each h_t . Greedy: predict argmax of logits.

Linear trick: if $g = \text{id}$, $h_t = W_{hh}^t h_0 + \sum_{j=1}^t W_{hh}^{t-j} W_{hx} x_j \Rightarrow$ classifier is linear in embeddings.

Scaled dot-product attention (Prob 2, 2023–25)

Form $X = E + P$ (token + positional), then

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$S = \frac{QK^\top}{\sqrt{d_k}}, \quad P = \text{softmax}(S), \quad Z = PV$$

Last token only: $s = q_t K^\top / \sqrt{d_k}$, $p = \text{softmax}(s)$, $z = pV$; logits = $W_{yz}z$, predict argmax.

Causal mask: set $S_{ij} = -\infty$ for $j > i$ before softmax (decoder).

Multi-head: concat heads then project W_O . Scaling a head's W_Q by c scales scores $\Rightarrow$ changes P and Z .

Math & ML foundations

- **Linear algebra:** inner product $x^\top y$; norms $\|x\|_2 = \sqrt{x^\top x}$, $\|x\|_1 = \sum |x_i|$, $\|x\|_\infty = \max_i |x_i|$. **Eigen:** $Av = \lambda v$; symmetric $A \Rightarrow$ real eigenvalues, orthogonal eigenvectors. **PSD** $A \succeq 0$ iff $x^\top A x \geq 0 \forall x$ iff all $\lambda_i \geq 0$. Gradient ∇f , Hessian $\nabla^2 f$; convex iff $\nabla^2 f \succeq 0$.
- **Probability:** Bayes $P(y|x) = P(x|y)P(y)/P(x)$. $\mathbb{E}[x]$, $\text{Var}(x) = \mathbb{E}[x^2] - \mathbb{E}[x]^2$. Entropy $H(p) = -\sum p \log p$; KL $\text{KL}(p||q) = \sum p \log(p/q) \geq 0$ (Gibbs). Gaussian $\mathcal{N}(\mu, \sigma^2)$.
- **ML:** learn $f: X \rightarrow Y$ from data (Mitchell: improve at task T on metric P with experience E). **Supervised** (labels) / **unsupervised** / **RL**. Regression (continuous y) vs classification (discrete y). Naive Bayes = estimate class-conditionals by counting (+ smoothing).
- **Over/underfit:** overfit = low train / high test error (high variance) $\rightarrow$ more data, regularize, dropout, early stop; underfit = both high (high bias) $\rightarrow$ more capacity/features. Test err $\approx$ bias² + variance + noise.

Softmax, sigmoid & temperature

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \in (0, 1), \quad \sum_i \text{softmax}(z)_i = 1$$

- For **finite** logits every component is **strictly** in $(0, 1)$ — never exactly 0/1 (since $e^x > 0$); those are reached only in a **limit**.
- **Shift-invariance:** $\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$ (the e^c cancels). Used for numerical stability: subtract $\max_i z_i$ first.
- **Temperature** $\text{softmax}(z/T)$, $T > 0$: $T \rightarrow 0^+ \rightarrow$ one-hot at argmax z ; $T \rightarrow \infty \rightarrow$ **uniform**. For finite T with **distinct** logits, output is strictly between the two; it is exactly uniform iff **all logits are equal** (then any T gives uniform).
- **Jacobian:** $\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$. With cross-entropy target y : $\partial L / \partial z = p - y$.

- **Sigmoid** = 2-class softmax: $\sigma(z) = \text{softmax}([z, 0])_1 = 1/(1 + e^{-z})$. Facts: $\sigma(-z) = 1 - \sigma(z)$, $\sigma(0) = 0.5$, $\sigma' = \sigma(1 - \sigma) \leq 0.25 \Rightarrow$ vanishing-gradient contributor).

Losses, MLE & MAP

- **Squared loss** = Gaussian-noise MLE; **cross-entropy/NLL** = categorical MLE. Min NLL = $\min \text{KL}(\hat{p}_{\text{data}} || p_\theta)$.
- **MAP** = MLE + log-prior = regularized MLE: Gaussian prior $\rightarrow \ell_2$ (ridge/weight decay), Laplace prior $\rightarrow \ell_1$ (sparsity).
- Cross-entropy $H(p, q) = H(p) + \text{KL}(p||q)$. Jensen: $\mathbb{E}[f(x)] \geq f(\mathbb{E}[x])$ for convex f .
- Why CE not MSE for classification: CE is convex in the logits, punishes confident-wrong hard, and gives calibrated probabilities; MSE+sigmoid saturates (near-zero gradient).
- Common losses: MSE (regression), MAE (robust), cross-entropy/NLL (classification), hinge $\max(0, 1 - yf)$ (margin/SVM). Convex in linear params: squared, logistic, hinge.

Linear regression & regularization

- Model $\hat{y} = w^\top x$ (bias via $x_0 = \{ \}$). Squared loss $\frac{1}{2} \|Xw - y\|^2$; **normal equations** $\hat{w} = (X^\top X)^{-1} X^\top y$. This LS solution = MLE under Gaussian noise.
- **Ridge** (ℓ_2) $\hat{w} = (X^\top X + \lambda I)^{-1} X^\top y$ = MAP with Gaussian prior; shrinks weights, always invertible for $\lambda > 0$, reduces variance. **Lasso** (ℓ_1) $\rightarrow$ sparse weights (feature selection), no closed form.
- **SGD** step (one example): $w \leftarrow w - \eta(w^\top x - y)x$. Minibatch trades variance (noise) vs compute.
- **Feature maps** $\varphi(x)$ let a linear model fit nonlinear targets (e.g. polynomial); one-hot encode categoricals.

Linear classifiers: perceptron, logistic, softmax

- **Perceptron:** predict $\text{sign}(w^\top x)$; on a mistake $y(w^\top x) \leq 0$ do $w \leftarrow w + yx$. **Novikoff:** if separable with unit-norm w^* , margin $\gamma = \min_i y_i w^{*\top} x_i > 0$, $\|x_i\| \leq R$, then the number of mistakes is $\leq (R/\gamma)^2$ (independent of N , dim). $\Rightarrow$ 100 mistakes on unit-norm data $\Rightarrow \gamma \leq 0.1$.
- **Logistic regression:** $P(y = \{ \} 1 | x) = \sigma(w^\top x)$; binary cross-entropy; $\nabla_w L = (\sigma(w^\top x) - y)x$. Loss convex $\Rightarrow$ unique min if **not separable**; on **separable** data the min is at $\|w\| \rightarrow \infty$ (no finite minimizer) — ℓ_2 restores one. Unlike the perceptron, GD gives **no finite-step** separation guarantee.
- **Softmax/multinomial regression:** $P(y = \{ \} k | x) = \text{softmax}(Wx)_k$; $\nabla_w L = (p_k - \mathbf{1}[y = \{ \} k])x$. Outputs live on the simplex.
- **Separability:** AND, OR are linearly separable; **XOR is not** (needs a hidden layer).

Neural networks & expressivity

- MLP: $h^{(\ell)} = g(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)})$, output layer softmax/linear. Nonlinearity essential — stacked linear layers collapse to one linear map.
- **Universal approximation:** one hidden layer with a non-polynomial activation approximates any continuous f on a compact set arbitrarily well — but width can be **exponential**; **depth** gives exponential efficiency for many functions. XOR needs only 1 hidden layer (2 ReLU units).
- **Activation trade-offs:** sigmoid/tanh **saturate** (grad $\rightarrow 0$); tanh is zero-centered (better-conditioned than sigmoid); ReLU is cheap/sparse, no positive saturation, but **dead** if pre-activation always < 0 ; Leaky/PreLU/ELU/GELU keep gradient for $z < 0$; GELU/SiLU are smooth (Transformers).

Activation derivatives

$g(z)$	$g'(z)$
$\sigma(z) = 1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z))$
$\tanh(z)$	$1 - \tanh^2(z)$
$\text{ReLU } \max(0, z)$	$\mathbf{1}[z > 0]$
Softplus $\log(1 + e^z)$	$\sigma(z)$
ELU ($z; e^z - 1$)	$\mathbf{1}; e^z = \min(e^z, 1)$
Leaky ReLU	$\mathbf{1}; \alpha$

Backpropagation & autodiff

Layer $z^{(\ell)} = W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}$, $h^{(\ell)} = g(z^{(\ell)})$. With $\delta^{(\ell)} = \partial L / \partial z^{(\ell)}$:

$$\delta^{(\ell)} = (W^{(\ell+1)\top} \delta^{(\ell+1)}) \odot g'(z^{(\ell)}), \quad \delta^{(L)} = \partial L / \partial h^{(L)} \odot g'(z^{(L)})$$

$$\partial L / \partial W^{(\ell)} = \delta^{(\ell)} (h^{(\ell-1)})^\top, \quad \partial L / \partial b^{(\ell)} = \delta^{(\ell)}, \quad \partial L / \partial h^{(\ell-1)} = W^{(\ell)\top} \delta^{(\ell)}$$

- Backprop = **reverse-mode autodiff**: one forward pass caches activations, one backward pass reuses shared sub-gradients $\Rightarrow$ cost $\approx 2 \times$ forward.
- Chain-rule MCQ: multiply local derivatives along each path, **sum** over paths.
- **Key gradients:** softmax+CE $\partial L / \partial z = p - y$; sigmoid+BCE $\sigma(z) - y$; MSE $\hat{y} - y$; linear $z = \{ \} W h \Rightarrow \partial L / \partial W = \delta h^\top$.

Optimization, initialization & regularization

- Optimizers:** SGD $\theta \leftarrow \theta - \eta \nabla$; **Momentum** $v \leftarrow \beta v - \eta \nabla$, $\theta \leftarrow \theta + v$; Nesterov (look-ahead); AdaGrad ($1/\sqrt{\sum g^2}$ per-param, decays LR); RMSProp (EMA of g^2); **Adam** = momentum + RMSProp + bias-corrected $\hat{m}, \hat{v}$: $\theta \leftarrow \theta - \eta \hat{m}/(\sqrt{\hat{v}} + \epsilon)$. LR schedules: warmup (Transformers), step/cosine decay. Minibatch: large = smoother/parallel/more memory; small = noisier (regularizing).
- Init** (variance preservation): **Xavier/Glorot** $\text{Var}(W) = 2/(n_{\text{in}} + n_{\text{out}})$ (tanh); **He** $2/n_{\text{in}}$ (ReLU). **Zero/equal init** = **symmetry problem**: all neurons get identical gradients and never differentiate.
- Regularization:** ℓ_2 /weight decay ($+\frac{\lambda}{2} \|w\|^2 \Rightarrow \text{grad} + \lambda w$); $\ell_1 \rightarrow$ sparse; **dropout** (zero units w.p. p at train, scale $1/(1-p)$; off at test; $\approx$ ensembling); data augmentation; early stopping; **label smoothing** (softens targets $\rightarrow$ better calibration).
- Vanishing/exploding gradients:** products of Jacobians shrink/grow. Fix: ReLU, good init, residuals, normalization, and **gradient clipping** $g \leftarrow g \cdot \min(1, \tau/\|g\|)$ (clipping fixes **exploding** only). Loss is **non-convex**; in high-dim **saddle points** dominate — SGD noise helps escape.

Normalization

- BatchNorm:** $\hat{x} = (x - \mu_B)/\sqrt{\sigma_B^2 + \epsilon}$, then $\gamma \hat{x} + \beta$. Smooths the landscape, allows higher LR, mild regularizer; **batch-size dependent**; uses running stats at test (train $\neq$ test behavior).
- LayerNorm:** normalize over **features per example** — batch-independent, standard in RNNs/Transformers. **RMSNorm** = LayerNorm without mean subtraction.

Convolutional networks

- Convolution = sparse, weight-shared linear map (cross-correlation): each output = $\sum (\text{filter} \odot \text{patch}) + b$. **Output size** $M' = \lfloor (M - k + 2P)/S \rfloor + 1$; with dilation d , effective kernel $k_e = d(k - \{ \} 1) + 1$.
- Params** ($k^2 C_{\text{in}} + 1$) C_{out} : pooling has none; conv $\ll$ FC in params (sharing + locality).
- Equivariance:** shift in $\rightarrow$ shift out (translation only, **not** rotation/scale). **Invariance:** conv + **pooling** $\rightarrow$ unchanged under small shifts. **Inductive bias:** translation equivariance + local receptive fields + parameter sharing. **Receptive field** grows with depth/stride/dilation.
- Backprop** through conv is **also a convolution** (flipped kernel); max-pool routes gradient to the arg-max cell only, avg-pool spreads equally.
- Building blocks:** 1×1 conv (per-pixel channel mixing / bottleneck); dilated (larger field, same params); transposed (upsampling); depthwise-separable (MobileNet, fewer params); global avg pool (replaces FC head). **ResNet** skip $y = \mathcal{F}(x) + x \Rightarrow \partial y / \partial x = \mathcal{F}'(x) + 1$ (gradient highway, enables very deep nets — **not** to skip layers).
- Architectures:** LeNet $\rightarrow$ AlexNet (ReLU+dropout+GPU, 2012) $\rightarrow$ VGG (stacked 3×3) $\rightarrow$ GoogLeNet/Inception (parallel multi-scale + 1×1 bottlenecks) $\rightarrow$ ResNet (skip, very deep).

Recurrent networks & BPTT

- $h_t = g(W_{hh} h_{t-1} + W_{hx} x_t + b)$, output $o_t = W_{yh} h_t$. **Parameter sharing** across time; trained by **BPTT** (unroll then backprop, summing gradients).
- Vanishing/exploding:** $\partial h_t / \partial h_k = \prod_{j=k+1}^t W_{hh}^T \text{diag}(g'(z_j))$ — governed by the **spectral radius** of W_{hh} (and $|g'| \leq 1$ for tanh/sigmoid): < 1 vanish, > 1 explode.
- LSTM:** gates $f, i, o, \sigma = \sigma(\cdot)$, $\tilde{c}_t = \tanh(\cdot)$, cell $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (additive path $\Rightarrow$ near-constant gradient), $h_t = o_t \odot \tanh(c_t)$. **GRU:** update+reset gates, no separate cell state (fewer params). **BiLSTM:** reads both directions.
- Autoregressive RNN makes **no Markov assumption** (full history in h_t). **Teacher forcing** (feed gold prev token) speeds training but causes **exposure bias**. Variants: recursive nets / Tree-LSTMs (tree structure), PixelRNN (images).

Sequence-to-sequence & attention

- Encoder-decoder:** encode source $\rightarrow$ decode target. A single fixed context vector is an **information bottleneck** $\rightarrow$ **attention** lets the decoder read **all** encoder states. Background: SMT / noisy-channel, IBM word alignments.
- Attention:** query s (decoder state), keys/values h_j (encoder). Scores $\rightarrow$ softmax weights $\alpha_j \rightarrow$ context $c = \sum_j \alpha_j h_j$. Score variants: **additive** (Bahdanau) $v^T \tanh(W_1 q + W_2 k)$; **dot/scaled-dot** (Luong) $q^T k / \sqrt{d}$.
- Decoding:** greedy (argmax); **beam search** (keep top- B , **length-normalize** to avoid short bias); sampling with temperature / top- k / top- p (nucleus). **BLEU** = geometric mean of n -gram precisions $\times$ brevity penalty; LM $\rightarrow$ perplexity.

Transformers & self-attention

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad Q=\{ \} XW_Q, K=\{ \} XW_K, V=\{ \} XW_V$$

- Why $\sqrt{d_k}$:** iid unit-variance q, k give $q \cdot k$ variance d_k ; dividing by $\sqrt{d_k}$ restores unit variance so softmax stays out of the low-gradient saturated regime.

- Multi-head:** h heads on d/h -dim subspaces, concat + W_O — attends to different subspaces/relations. Params/block $\approx 4d^2$ (Q,K,V,O) + FFN $\approx 8d^2$. **Block:** (multi-head self-attn $\rightarrow$ residual+LayerNorm $\rightarrow$ position-wise FFN $\approx 4 \times$ wider $\rightarrow$ residual+LayerNorm); **pre-LN** trains more stably than post-LN.
- Positional encoding** needed (self-attention is permutation-equivariant): sinusoidal $\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d})$ (relative offsets = linear maps, extrapolates) or learned. Deliberately omitted for permutation-invariant tasks.
- Masking:** causal/decoder sets future scores to $-\infty$ **before** softmax (autoregressive); padding masks ignore pad tokens. Decoder also has **cross-attention** to the encoder.
- Complexity:** self-attention $O(L^2 d)$ but **parallel** over positions; RNN $O(Ld^2)$ but **sequential**.

Representation learning & embeddings

- Autoencoder:** encoder $\rightarrow$ bottleneck $\rightarrow$ decoder, minimize reconstruction (MSE). **Linear AE + squared loss = PCA:** optimum spans the top-variance subspace (via SVD of centered X / Eckart-Young). Variants: denoising, sparse, stochastic AEs; used for pretraining/transfer.
- PCA:** top eigenvectors of the covariance maximize retained variance (equivalently top singular vectors).
- word2vec: skip-gram** (predict context from word) / **CBOW** (predict word from context); **negative sampling** replaces the full softmax with $\log \sigma(u_i^T v_w) + \sum_{\text{neg}} \log \sigma(-u_i^T v_w)$. Captures analogies (king — man + woman $\approx$ queen). GloVe factorizes log co-occurrence; FastText adds sub-words.
- Static (word2vec/GloVe) = one vector/word; **contextual** (ELMo/BERT) = vector depends on the sentence.

Self-supervised & pretrained models

- Self-supervised pretraining:** learn from unlabeled text via a proxy objective, then fine-tune. **BPE** tokenization iteratively merges the most frequent adjacent pair $\Rightarrow$ open-vocabulary subwords (rare words = sequences of subwords).
- BERT:** encoder-only, **masked LM** (mask 15%), bidirectional [+ next-sentence pred] $\rightarrow$ understanding/classification. **GPT:** decoder-only, causal/next-token LM $\rightarrow$ generation. **T5/BART:** encoder-decoder, span corruption (text-to-text).
- Adaptation:** full fine-tune vs parameter-efficient — **adapters** (small bottleneck layers), **LoRA** ($W + BA$, rank $r \ll d$, train only B, A), prompt/prefix-tuning. **In-context learning** = few/zero-shot, **no weight update**.
- Scaling laws:** loss falls as a power law in params/data/compute (compute-optimal balances them); emergent abilities at scale. **RLHF/RLVR:** SFT $\rightarrow$ reward model from human preferences $\rightarrow$ PPO; aligns behavior ($\neq$ capability). Risks: “stochastic parrots” / hallucination / bias.

Generative models

- Autoregressive** $p(x) = \prod_t p(x_t | x_{<t})$: exact likelihood, slow sequential sampling (PixelRNN, GPT).
- VAE:** encoder outputs a **distribution** $q(z|x) = \mathcal{N}(\mu, \sigma^2)$. Maximize the **ELBO** (lower bound on $\log p(x)$; gap = $\text{KL}(q \| \text{true posterior})$):

$$\mathcal{L} = \underbrace{\mathbb{E}_q[\log p(x|z)]}_{\text{reconstruction}} - \underbrace{\text{KL}(q(z|x) \| p(z))}_{\text{regularizer}}$$

Reparameterization $z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ (differentiable sampling). $\text{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0, I)) = \frac{1}{2} \sum_i (\sigma_i^2 + \mu_i^2 - 1 - \ln \sigma_i^2)$. Risk: **posterior collapse**.

- GAN:** $\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$; implicit density, sharp samples, **mode collapse** / instability; non-saturating & Wasserstein (WGAN) losses stabilize.
- Diffusion:** forward adds Gaussian noise over T steps; model learns to denoise (predict ϵ). High quality, slow sampling. **Energy-based:** $p(x) \propto e^{-E(x)}$. **Normalizing flows:** invertible maps, **exact** likelihood via change-of-variables.

Model	Likelihood / sampling / failure
Autoregressive	exact likelihood; slow sequential sampling
VAE	ELBO (approx.); fast; posterior collapse
GAN	implicit (none); fast; mode collapse / unstable
Diffusion	approx.; slow (iterative); high quality
Norm. flow	exact; fast; architecture-constrained

Uncertainty, explainability & fairness

- Aleatoric** uncertainty = data noise (irreducible); **epistemic** = model uncertainty (reducible with data). Total = aleatoric + epistemic; predictive entropy = total, mutual information (BALD) isolates epistemic. Estimate epistemic via **deep ensembles**, **MC-dropout**, BNNs. **Distribution shift** (covariate/label/concept) inflates epistemic uncertainty.
- Calibration:** confidence $\approx$ accuracy. $\text{ECE} = \sum_i (n_i/N) |\text{acc}(b_i) - \text{conf}(b_i)|$ over confidence bins; reliability diagram plots conf vs acc; **temperature scaling** (one logit param) is a post-hoc fix.

- **Explainability**: **intrinsic** (interpretable models) vs **post-hoc**. Gradient-based: saliency, integrated gradients, Grad-CAM. Surrogates: LIME, SHAP; counterfactuals. Attention-as-explanation is **contested**.
- **Fairness**: bias originates in data (e.g. co-occurrence imbalance $\rightarrow$ MT gender bias, fixed by data augmentation). Metrics: **demographic parity** ($P(\hat{y}=\{1\}|A)$ equal), **equalized odds** (equal TPR and FPR), **equal opportunity** (equal TPR) — **mutually incompatible in general** (impossibility). Also: privacy (membership inference / PII), sustainability (compute cost). WEAT measures embedding bias.

Information theory & metrics

- $H(p) = -\sum p \log p$; cross-entropy $H(p, q) = -\sum p \log q$; $\text{KL}(p\|q) = H(p, q) - H(p) \geq 0$. **Perplexity** $= e^{\text{NLL}}$ (LM, lower better).
- Classification: $\text{Prec} = \text{TP}/(\text{TP} + \text{FP})$, $\text{Rec} = \text{TP}/(\text{TP} + \text{FN})$, $F_1 = 2 \text{PR}/(P + R)$, $\text{accuracy} = (\text{TP} + \text{TN})/\text{all}$; ROC-AUC. Generation: BLEU (MT), perplexity (LM).

Generalization & double descent

- Test error $\approx \text{bias}^2 + \text{variance} + \text{noise}$: high bias = underfit, high variance = overfit; regularization trades variance for bias.
- Deep nets show **double descent**: past the interpolation threshold test error can **fall again**. Overparameterized nets generalize despite fitting noise — attributed to SGD's **implicit regularization** (bias toward low-norm / flat minima). Classical capacity bounds (VC / Rademacher) are loose here. More parameters than samples is fine.

! “False-statement” traps

1. GD for logistic reg finds a separator in finite steps — **FALSE** (perceptron does).
2. Masking is used in the **encoder** to block future tokens — **FALSE** (decoder; future = output seq).
3. GPT-3 is encoder-only masked-LM — **FALSE** (decoder-only, causal).
4. Gradient **clipping** prevents **vanishing** grads — **FALSE** (prevents exploding).
5. Adding a constant to softmax logits changes output — **FALSE** (shift-invariant); scaling **does**.
6. Convolution is **rotation**-equivariant — **FALSE** (translation only).
7. Autoregressive RNN makes a **Markov** assumption — **FALSE** (full history via h_t).
8. Skip connections exist to **drop/skip layers** — **FALSE** (gradient flow).
9. Linear AE differs from PCA — **FALSE** (equivalent).
10. More params than samples $\Rightarrow$ can't generalize — **FALSE** (deep nets often do).
11. VAE encodes to a fixed point — **FALSE** (a distribution).
12. BatchNorm behaves the same at train and test — **FALSE** (batch vs running stats).

Worked Exam Problems — from past exams (official solutions)

R1 • CNN forward pass + equivariance / invariance (2024, Prob 1)

Conv layer, 3×3 filter, stride 1, pad 1, ReLU, bias $b = 0$:

$$K = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{pmatrix}, \quad x = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

(a) Compute the layer output h . (b) Repeat for the input translated one column right. (c) Apply a 2×2 max-pool (stride 2) to each; identify the property.

Solution. (a) Zero-pad x to 6×6 (a ring of zeros). Each output cell = sum of the elementwise product of K with the 3×3 padded window whose centre is that pixel.

- Cell $z_{1,1}$ (top-left): window $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, so $z_{1,1} = 1 \cdot (-1) = -1$ (only the bottom-right entry is non-zero, aligned with $K_{3,3}\{=\}\{-1\}$).
- Cell $z_{3,3}$: window $\begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$, so $z_{3,3} = 1 \cdot 1 + 1 \cdot 2 + 1 \cdot 1 = 4$ (the “1” column meets K ’s first column $[1, 2, 1]$).

Doing all 16 cells:

$$z = \begin{pmatrix} -1 & -2 & 1 & 2 \\ -3 & -1 & 3 & 1 \\ -4 & 0 & 4 & 0 \\ -3 & 0 & 3 & 0 \end{pmatrix}, \quad h = \text{ReLU}(z) = \begin{pmatrix} 0 & 0 & 1 & 2 \\ 0 & 0 & 3 & 1 \\ 0 & 0 & 4 & 0 \\ 0 & 0 & 3 & 0 \end{pmatrix}$$

(ReLU zeros every negative entry.) (b) Translating the input one column right translates the whole output one column right $\Rightarrow$ **equivariance**: $f(\text{shift}(x)) = \text{shift}(f(x))$. (c) Max-pool of h with 2×2 windows (stride 2): top-left window $\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \rightarrow 0$, top-right $\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix} \rightarrow 3$, bottom-left $\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \rightarrow 0$, bottom-right $\begin{pmatrix} 4 & 0 \\ 0 & 3 \end{pmatrix} \rightarrow 4$, giving $\begin{pmatrix} 0 & 3 \\ 0 & 4 \end{pmatrix}$. The translated input yields the **same** pooled output $\begin{pmatrix} 0 & 3 \\ 0 & 4 \end{pmatrix} \Rightarrow$ **invariance** (conv + pooling is approximately shift-invariant).

R2 • CNN parameter table + Inception forward (2023, Prob 1, GoogLeNet)

A 1×1 conv on an $H \times W \times C$ image applies a filter of shape $1 \times 1 \times C$ (a per-pixel FC over channels; used to change channel count). For an Inception block with single-output-channel convs, stride 1, pad 0 unless stated:

Element	# params	Dimension
h_1 (1×1 conv)	$C + 1$	$H \times W$
h_2 (1×1 conv)	$C + 1$	$H \times W$
h'_2 (3×3 conv, pad 1)	$3 \cdot 3 + 1 = 10$	$H \times W$
h_3 (1×1 conv)	$C + 1$	$H \times W$
h'_3 (5×5 conv, pad 2)	$5 \cdot 5 + 1 = 26$	$H \times W$
h_4 (3×3 maxpool, pad 1)	0	$H \times W \times C$
h'_4 (1×1 conv)	$C + 1$	$H \times W$
h (concat)	—	$H \times W \times 4$

Forward: the 1×1 conv $K = [0.5, 0.5]$ averages the 2 channels $\rightarrow h_2 = \begin{pmatrix} 1.0 & 1.0 & 1.0 \\ 0.8 & 0.5 & 0.8 \\ 0.9 & 0.8 & 0.9 \end{pmatrix}$. Then 3×3 conv $K_{3 \times 3} = \begin{pmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{pmatrix}$ (pad 1):

Solution. $K_{3 \times 3}$ has -1 on its top row, 0 middle, $+1$ bottom $\Rightarrow$ each output = (sum of bottom row of window) – (sum of top row).

- Cell $z'_{2,(1,1)}$ (pad the 3×3 h_2 to 5×5 ; window = top-left corner) = $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0.8 & 0.5 \\ 0 & 0.9 & 0.8 \end{pmatrix}$: bottom row $0 + 0.8 + 0.5 = 1.3$ minus top row $0 \Rightarrow 1.3$.
- Cell $z'_{2,(2,1)}$: window $\begin{pmatrix} 0 & 1.0 & 1.0 \\ 0 & 0.8 & 0.5 \\ 0 & 0.9 & 0.8 \end{pmatrix}$: bottom 1.7 – top $2.0 = -0.3$.

$$z'_2 = \begin{pmatrix} 1.3 & 2.1 & 1.3 \\ -0.3 & -0.4 & -0.3 \\ -1.3 & -2.1 & -1.3 \end{pmatrix}, \quad h'_2 = \text{ReLU}(z'_2) = \begin{pmatrix} 1.3 & 2.1 & 1.3 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

R3 • RNN forward pass + greedy prediction (2024, Prob 2)

Autoregressive RNN, $h_t = \text{ReLU}(W_{hx}x_t + W_{hh}h_{t-1})$, all biases 0, $h_0 = \mathbf{0}$, with

$$W_{hx} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & -1 & -1 \end{pmatrix}, \quad W_{hh} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Sentence “Awful plot and actors. The movie is” with embeddings $x_{\text{Awful}} = [0, -5, 0]^\top$, $x_{\text{plot}} = [1, 0, 0]^\top$, $x_{\text{actors.}} = [-1, 0, 0]^\top$, $x_{\text{movie}} = [-2, 0, 0]^\top$, others $[0, 0, 0]^\top$. Output row $w_{\text{good}} = [1, -1]$, $w_{\text{bad}} = [-1, 1]$.

Solution. Since $W_{hh} = I$, each step adds $W_{hx}x_t$ and re-applies ReLU:

$$h_1 = [0, 0], \quad h_2 = [0, 5], \quad h_3 = [1, 5], \quad h_4 = [1, 5], \quad h_5 = [0, 5], \quad \dots \quad h_8 = [0, 5].$$

E.g. $h_2 = \text{ReLU}(W_{hx}[0, -5, 0]^\top + 0) = \text{ReLU}([-5, 5]^\top) = [0, 5]^\top$. Logits at the “[Z]” slot: $\text{good} = w_{\text{good}}h_8 = -5$, $\text{bad} = w_{\text{bad}}h_8 = 5$. Since $\text{bad} > \text{good} \Rightarrow$ predict **negative** sentiment. (With **linear** activations, unrolling shows the classifier becomes **linear** in the embeddings.)

R4 • Self-attention, last-token shortcut (2024, Prob 2, transformer)

Single self-attention head, $W_Q = W_K = W_V = \begin{pmatrix} 0.1 & 0 \\ 0 & -0.1 \\ 0 & 0.1 \end{pmatrix}$, scaled dot-product with $\sqrt{d_k} = \sqrt{2}$. You only need the **last** token’s query $q_8 = [0, 0.7]$ and all keys.

Solution.

$$s_8 = q_8^\top K^\top / \sqrt{2} = \frac{1}{\sqrt{2}} [0, 0.42, 0.14, 0.21, 0.28, 0.35, 0.42, 0.49]$$

$$p_8 = \text{softmax}(s_8) = [0.101, 0.136, 0.112, 0.118, 0.124, 0.130, 0.136, 0.143]$$

$$z_8 = p_8^\top V = [-0.028, 0.436]$$

Logits: good = $[1, -1]z_8 = -0.464$, bad = $[-1, 1]z_8 = 0.464 \Rightarrow$ predict **negative**. **Trick:** for the last-token prediction you never need the full P matrix — just its last row.

🔥 Practice Problems — my own (exam-style, with worked solutions)

M1 · CNN output sizes & parameter count

A CNN takes a $32 \times 32 \times 3$ image: Conv1 (16 filters 3×3 , stride 1, pad 1) $\rightarrow$ MaxPool (2×2 , stride 2) $\rightarrow$ Conv2 (32 filters 3×3 , stride 1, pad 1) $\rightarrow$ MaxPool (2×2 , stride 2) $\rightarrow$ Flatten $\rightarrow$ FC (10 outputs). Give each layer's output size and trainable-parameter count.

Solution. Output size $M' = \lfloor \frac{M-k+2P}{S} \rfloor + 1$; conv params $(k \cdot k \cdot C_{\text{in}} + 1)C_{\text{out}}$.

Layer	Output	Params
Conv1	$32 \times 32 \times 16$	$(3 \cdot 3 \cdot 3 + 1) \cdot 16 = 448$
MaxPool	$16 \times 16 \times 16$	0
Conv2	$16 \times 16 \times 32$	$(3 \cdot 3 \cdot 16 + 1) \cdot 32 = 4640$
MaxPool	$8 \times 8 \times 32$	0
Flatten	2048	0
FC	10	$(2048 + 1) \cdot 10 = 20490$

Total = 448 + 4640 + 20490 = 25578 parameters. (Conv sizes unchanged by “same” padding $P = 1, k = 3$; pooling halves each spatial dim.)

M2 · Scaled dot-product self-attention (last token)

Three tokens, $d_k = 2$, identity projections ($Q = K = V = X$), $\sqrt{d_k} = \sqrt{2}$:

$$X = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 2 & 1 \end{pmatrix} \quad (x_1, x_2, x_3 \text{ as rows}).$$

Compute the context vector z_3 for the last token, and predict good vs bad with $w_{\text{good}} = [1, -1]$, $w_{\text{bad}} = [-1, 1]$.

Solution. Query $q_3 = [2, 1]$. Raw scores $q_3 \cdot k_i$: $k_1\{\cdot\}, 2 \cdot 2 + 1 \cdot 0 = 4$, $k_2\{\cdot\}, 0 + 2 = 2$, $k_3\{\cdot\}, 4 + 1 = 5$; scaled by $1/\sqrt{2}$: $[2.828, 1.414, 3.536]$. Softmax = exponentiate and normalize: $e^{2.828}\{\cdot\}16.92$, $e^{1.414}\{\cdot\}4.11$, $e^{3.536}\{\cdot\}34.33$; sum = 55.36:

$$p_3 = [16.92, 4.11, 34.33]/55.36 = [0.306, 0.074, 0.620].$$

$$z_3 = 0.306[2, 0] + 0.074[0, 2] + 0.620[2, 1] = [0.611\{+\}1.240, 0.148\{+\}0.620] = [1.851, 0.769].$$

Logits: good = $[1, -1] \cdot z_3 = 1.851 - 0.769 = 1.082 >$ bad = $-1.082 \Rightarrow$ predict **good**. (The last token attends most to itself, score 5.)

M3 · Backpropagation through a 2-layer MLP

$x = [1, 2]$, hidden $h = \text{ReLU}(W_1 x)$, output $\hat{y} = w_2^\top h$, loss $L = \frac{1}{2}(\hat{y} - y)^2$, target $y = 1$, with

$$W_1 = \begin{pmatrix} 1 & -1 \\ 0 & 1 \end{pmatrix}, \quad w_2 = [1, 1].$$

Compute all gradients and one SGD step ($\eta = 0.1$).

Solution. Forward: $z_1 = W_1 x = [-1, 2]$, $h = \text{ReLU}(z_1) = [0, 2]$, $\hat{y} = 1 \cdot 0 + 1 \cdot 2 = 2$, $L = \frac{1}{2}(2 - 1)^2 = 0.5$. **Backward:** $\partial L / \partial \hat{y} = \hat{y} - y = 1$.

$$\partial L / \partial w_2 = (\hat{y} - y)h = [0, 2]; \quad \partial L / \partial h = (\hat{y} - y)w_2 = [1, 1].$$

$$\partial L / \partial z_1 = \partial L / \partial h \odot \text{ReLU}'(z_1) = [1, 1] \odot [0, 1] = [0, 1].$$

$$\partial L / \partial W_1 = (\partial L / \partial z_1)x^\top = \begin{pmatrix} 0 & 0 \\ 1 & 2 \end{pmatrix}.$$

SGD: $w_2 \leftarrow [1, 0.8]$, $W_1 \leftarrow \begin{pmatrix} 1 & -1 \\ -0.1 & 0.8 \end{pmatrix}$.

M4 · Logistic-regression SGD update

Binary logistic regression, $P(y\{=\}1|x) = \sigma(w^\top x + b)$, one step of SGD on the NLL loss with $\eta = 1$, from $w = [0, 0]$, $b = 0$, on $(x, y) = ([1, 2], 1)$.

Solution. $z = w^\top x + b = 0$, $\sigma(0) = 0.5$. Gradient $(\sigma - y)x = (0.5 - 1)[1, 2] = [-0.5, -1]$, and $(\sigma - y) = -0.5$ for b . Update: $w \leftarrow [0, 0] - 1 \cdot [-0.5, -1] = [0.5, 1]$, $b \leftarrow 0 - 1 \cdot 0.5 = -0.5$. The boundary moves toward classifying x as positive.

M5 · Multi-head attention — effect of the projection matrices (recurring MCQ + Prob 2)

Head 1 has $W_Q^{(1)}, W_K^{(1)}, W_V^{(1)}$. Relate P, Z of a second head in two cases: (a) negated $W_Q^{(2)}\{\cdot\}\{-\}W_Q^{(1)}, W_K^{(2)}\{\cdot\}\{-\}W_K^{(1)}, W_V^{(2)}\{\cdot\}\{-\}W_V^{(1)}$; (b) scaled $W_Q^{(2)}\{\cdot\}\{2\}W_Q^{(1)}, W_K^{(2)}\{\cdot\}\{2\}W_K^{(1)}, W_V^{(2)}\{\cdot\}\{\frac{1}{2}\}W_V^{(1)}$.

Solution. (a) $Q^{(2)}K^{(1)\top} = (-Q^{(1)})(-K^{(1)})^\top = Q^{(1)}K^{(1)\top} \Rightarrow P^{(2)} = P^{(1)}$ (signs cancel **inside** softmax). But $Z^{(2)} = P^{(2)}V^{(2)} = P^{(1)}(-V^{(1)}) = -Z^{(1)}$. So $P^{(1)}\{\cdot\}P^{(2)}, Z^{(1)}\{\cdot\}Z^{(2)}$. (b) Scores scale by 2: $P^{(2)} = \text{softmax}(2Q^{(1)}K^{(1)\top}/\sqrt{d}) \neq P^{(1)}$ (softmax is **not** scale-invariant). And $Z^{(2)} = P^{(2)}(\frac{1}{2}V^{(1)}) \neq Z^{(1)}$. So **both** differ.

Rule: a common factor on **both** Q, K that cancels (like $-1 \cdot -1$) leaves P unchanged; any **net** scaling of $QK^\top$ changes P .

M6 · Causal (masked) self-attention matrix

Raw scaled scores $S = \begin{pmatrix} 2 & 4 & 1 \\ 1 & 3 & 5 \\ 0 & 2 & 4 \end{pmatrix}$ (row i = query i). Apply a **causal** mask (token i attends only to $j \leq i$) then row-softmax to get P .

Solution. Set upper triangle to $-\infty$ **before** softmax ($e^{-\infty}\{\cdot\}0$), then softmax each row over the allowed entries.

- Row 1: only $j\{=\}1$ allowed $\rightarrow [1, 0, 0]$.
- Row 2: softmax($[1, 3]$): $e^1\{\cdot\}2.72$, $e^3\{\cdot\}20.09$, sum 22.81 $\rightarrow [2.72, 20.09]/22.81 = [0.119, 0.881]$.
- Row 3: softmax($[0, 2, 4]$): $e^0\{\cdot\}1$, $e^2\{\cdot\}7.39$, $e^4\{\cdot\}54.60$, sum 62.99 $\rightarrow [0.016, 0.117, 0.867]$.

$$P = \begin{pmatrix} 1 & 0 & 0 \\ 0.119 & 0.881 & 0 \\ 0.016 & 0.117 & 0.867 \end{pmatrix}$$

P is lower-triangular $\Rightarrow$ no token sees the future (autoregressive/decoder).

M7 · BPTT — vanishing vs exploding gradient

Scalar RNN, linear activation, $h_t = w_{hh}h_{t-1} + w_{hx}x_t$. How does $\partial h_t / \partial h_{t-k}$ behave over k steps for $w_{hh} = 0.5$ vs $w_{hh} = 1.5$?

Solution. $\partial h_t / \partial h_{t-1} = w_{hh}$, so over k steps $\partial h_t / \partial h_{t-k} = w_{hh}^k$. $w_{hh} \approx 0.5$: $0.5^{10} \approx 0.001 \Rightarrow$ **vanishes**. $w_{hh} \approx 1.5$: $1.5^{10} \approx 57.7 \Rightarrow$ **explodes**. General (vector): factor is $\|W_{hh}^\top \text{diag}(g')\|$ per step; < 1 vanish, > 1 explode. LSTM/GRU's additive cell state ≈ 1 per step avoids this; clipping caps explosion.

M8 • VAE — ELBO with KL between Gaussians

Encoder outputs $q(z|x) = \mathcal{N}(\mu, \sigma^2)$ with $\mu = [1, 0]$, $\sigma^2 = [0.25, 1]$; prior $\mathcal{N}(0, I)$. Given reconstruction term $\mathbb{E}_{q[\log p(x|z)]} = -1.5$, compute the KL term and the ELBO.

Solution.

$$\text{KL} = \frac{1}{2} \sum_i (\sigma_i^2 + \mu_i^2 - 1 - \ln \sigma_i^2)$$

Dim 1: $\frac{1}{2}(0.25 + 1 - 1 - \ln 0.25) = \frac{1}{2}(0.25 + 1.386) = 0.818$. Dim 2: $\frac{1}{2}(1 + 0 - 1 - \ln 1) = 0$. So $\text{KL} = 0.818$.

$$\text{ELBO} = \underbrace{-1.5}_{\text{recon}} - \underbrace{0.818}_{\text{KL}} = -2.318.$$

Maximizing ELBO = better reconstruction + latent close to the prior. Sample via $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$.

M9 • Computation-graph chain rule (MCQ style)

Given $y = \sigma(ab + c)$ with $a \approx 1$, $b \approx 2$, $c \approx 0$, which is $\partial y / \partial a$? (a) $\sigma'(ab + c)$ (b) $y(1 - y)b$ (c) $y(1 - y)a$ (d) $y(1 - y)$

Solution. Answer (b). Chain rule: $\partial y / \partial a = \sigma'(z) \cdot \partial z / \partial a = y(1 - y) \cdot b$ (since $z = ab + c$, $\partial z / \partial a = b$). Numerically $z \approx 2$, $y = \sigma(2) = 0.881$, so $\partial y / \partial a = 0.881 \cdot 0.119 \cdot 2 = 0.210$.

M10 • Softmax temperature

Logits $z = [2, 1, 0]$. Give $\text{softmax}(z/T)$ for $T = 1, 0.5, 2$ and describe the effect.

Solution.

T	z/T	$e^{z/T} / \text{sum}$	softmax
1	[2, 1, 0]	[7.39, 2.72, 1]/11.11	[0.665, 0.245, 0.090]
0.5	[4, 2, 0]	[54.6, 7.39, 1]/62.99	[0.867, 0.117, 0.016]
2	[1, 0.5, 0]	[2.72, 1.65, 1]/5.37	[0.506, 0.307, 0.186]

Lower $T \rightarrow$ **sharper** ($\rightarrow \text{argmax}$); higher $T \rightarrow$ **flatter** ($\rightarrow \text{uniform}$). $T \rightarrow 0$: one-hot; $T \rightarrow \infty$: uniform. Adding a constant to z changes **nothing** (shift-invariance); scaling by $1/T$ **does**.

M11 • Perceptron — counting mistakes

Run the (no-bias) perceptron, $w \in [0, 0]$, on $x_1 \in [1, 0], (+1)$, $x_2 \in [0, 1], (+1)$, $x_3 \in [-1, -1], (-1)$, in order. How many mistakes to converge?

Solution. x_1 : $w \cdot x_1 = 0 \leq 0 \rightarrow$ mistake, $w \leftarrow w + x_1 = [1, 0]$. x_2 : $w \cdot x_2 = 0 \leq 0 \rightarrow$ mistake, $w \leftarrow w + x_2 = [1, 1]$. x_3 : $w \cdot x_3 = -2$, $y_3(w \cdot x_3) = (-1)(-2) = 2 > 0 \rightarrow$ correct. Re-check x_1, x_2 : both $> 0 \checkmark$. Converged after **2 mistakes**. (Guaranteed finite because the data is linearly separable — Novikoff.)

✓ Multiple-Choice Bank — exam-style themes (with answers)

Cover the ✓ Ans line and try each first. These mirror the recurring Part-1 MCQ themes.

Q1 (Softmax invariance). For $g(s, t) = \text{softmax}(vs + 1t)$ with $v \neq 0$ and scalars s, t : (a) constant in both s, t ; (b) constant in t , not s ; (c) constant in s , not t ; (d) constant in neither.

✓ **Ans.** (b). Adding $1t$ shifts all logits equally $\Rightarrow$ softmax unchanged in t (shift-invariance). Scaling by $s = \text{temperature} \Rightarrow$ output does change.

Q2 (Perceptron on scaled data). A separable set D and $D' = \{(2x, y)\}$ (inputs doubled). No bias, $w_0 = 0$. On which does the perceptron make more mistakes?

✓ **Ans.** Same for both. Updates on D' are $w\{+\}2yx =$ exactly twice those on D ; the sign of $w \cdot x$ at each step is identical, so the mistake sequence matches.

Q3 (ResNet). The main purpose of skip connections in ResNets is: (a) connect input directly to output; (b) skip layers to cut compute; (c) randomly drop layers like dropout; (d) let the gradient flow directly, mitigating vanishing gradients.

✓ **Ans.** (d). $y = \mathcal{F}(x) + x$ gives $\partial y / \partial x = \mathcal{F}'(x) + 1$, so gradient always has a path back.

Q4 (Multi-head, negated). Head 2 uses $W_Q^{(2)}\{=\}\{-\}W_Q^{(1)}$, $W_K^{(2)}\{=\}\{-\}W_K^{(1)}$, $W_V^{(2)}\{=\}\{-\}W_V^{(1)}$. Relation of P, Z ?

✓ **Ans.** $P^{(1)}\{=\}\{P^{(2)}\}$ and $Z^{(1)}\{=\}\{-\}Z^{(2)}$. Signs cancel in $QK^\top$ (so P equal) but survive in PV (so Z negated).

Q5 (Masked attention — pick the FALSE). (a) Decoder masking lets each token attend only to previous tokens; (b) masking is applied in the encoder to stop attending to later input tokens; (c) future tokens are hidden during training; (d) it is crucial for language modeling.

✓ **Ans.** (b) is FALSE. Causal masking lives in the decoder and refers to the output sequence; the encoder attends bidirectionally.

Q6 (VAE vs AE). What distinguishes a variational autoencoder? (a) different hidden activation; (b) encodes to fixed points; (c) introduces randomness — a distribution over the latent space per input; (d) only does dimensionality reduction.

✓ **Ans.** (c). The encoder outputs μ, σ defining $q(z|x) = \mathcal{N}(\mu, \sigma^2)$; sampling + the KL term regularize the latent space.

Q7 (Softplus derivative). $\frac{d}{dt} \log(1 + e^t) =$ (a) $\sigma(t)(1 - \sigma(t))$; (b) $\sigma(t)$; (c) $\sigma(-t)$; (d) $\text{softplus}(t)(1 - \text{softplus}(t))$.

✓ **Ans.** (b) $\sigma(t) = 1/(1 + e^{-t})$. Differentiate: $e^t/(1 + e^t) = \sigma(t)$.

Q8 (Commuting ops). Which pair always gives the same result when swapped? (a) conv & max-pool; (b) conv & ReLU; (c) ReLU & max-pool; (d) none.

✓ **Ans.** (c). ReLU is monotonic non-decreasing, so $\max(\text{ReLU}(\cdot)) = \text{ReLU}(\max(\cdot))$. Conv is linear and does not commute with either nonlinear op.

Q9 (Which layer?). Which computes its output via dot products between input patches and filters? (a) convolution; (b) max-pool; (c) fully-connected; (d) none.

✓ **Ans.** (a). That is the definition of a convolutional layer (cross-correlation of a filter over patches).

Q10 (Pretrained — pick the FALSE). (a) BERT is encoder-only, trained with masked LM; (b) GPT is decoder-only, causal LM; (c) GPT-3 is encoder-only with masked LM; (d) T5 is encoder-decoder.

✓ **Ans.** (c) is FALSE. GPT models are decoder-only, autoregressive — not encoder-only MLM.

Q11 (Gradient clipping — pick the FALSE). (a) clipping bounds the gradient norm; (b) it helps with exploding gradients in RNNs; (c) it prevents vanishing gradients; (d) it is common in BPTT.

✓ **Ans.** (c) is FALSE. Clipping caps large gradients (exploding); vanishing needs LSTM/GRU/init/residuals.

Q12 (Autoregressive RNN — pick the FALSE). (a) it conditions on the full history via h_t ; (b) it makes a Markov (fixed-window) assumption; (c) it can model long dependencies in principle; (d) it is trained by BPTT.

✓ **Ans.** (b) is FALSE. The recurrent state summarizes all past tokens — no fixed Markov window.

Q13 (Parallelism). Which is TRUE about training a Transformer vs an RNN? (a) both are sequential over positions; (b) self-attention can be computed in parallel across positions; (c) RNNs are $O(L^2)$; (d) attention is cheaper than $O(L^2)$.

✓ **Ans.** (b). Self-attention has no recurrence, so all positions compute together ($O(L^2d)$, parallel); the RNN must go step by step.

Q14 (Conv output size). Input 28×28 , filter 5×5 , stride 1, padding 0. Output spatial size? (a) 28; (b) 26; (c) 24; (d) 23.

✓ **Ans.** (c) 24. $M' = \left\lfloor \frac{28-5+0}{1} \right\rfloor + 1 = 24$.

Q15 (Parameter count). A conv layer: input $32 \times 32 \times 3$, 10 filters 5×5 . Trainable params? (a) 250; (b) 750; (c) 760; (d) 7680.

✓ **Ans.** (c) $760 = (5 \cdot 5 \cdot 3 + 1) \cdot 10$ (don't forget the +1 bias per filter).

Q16 (Diagnosis). A high-degree polynomial fits the training points perfectly but tests poorly. This is: (a) underfitting, add complexity; (b) overfitting, use ℓ_2 / more data; (c) underfitting, use dropout; (d) fine, no action.

✓ **Ans.** (b). Low train + high test error = overfitting (high variance) $\rightarrow$ regularize, get more data, or reduce capacity.

Q17 (Zero init — pick the FALSE). (a) zero-init makes all neurons in a layer receive identical gradients; (b) this is the symmetry problem; (c) it prevents the network from learning distinct features; (d) it is the recommended default init.

✓ **Ans.** (d) is FALSE. Zero init is bad (symmetry); use Xavier/Glorot or He.

Q18 (Attention scaling). Why divide the dot-product scores by $\sqrt{d_k}$? (a) to normalize to a probability; (b) to keep score variance ≈ 1 so softmax gradients don't vanish; (c) to make it permutation-invariant; (d) to reduce parameters.

✓ **Ans.** (b). For large d_k , unscaled dot products grow, pushing softmax into saturated regions with tiny gradients.

Q19 (Separability — pick the FALSE). (a) the perceptron converges in finite steps if data is separable; (b) gradient descent for logistic regression is guaranteed to find a separator in finite steps; (c) deep nets can have more params than samples and still generalize; (d) conv layers have fewer params than equivalent FC layers.

✓ **Ans.** (b) is FALSE. Logistic-regression GD has no finite-step separation guarantee (loss minimized only as $\|w\| \rightarrow \infty$).

Q20 (Linear autoencoder). A single-hidden-layer autoencoder with linear activations and squared-error loss learns: (a) the same subspace as PCA; (b) a strictly better code than PCA; (c) a random projection; (d) nothing useful.

✓ **Ans.** (a). Its optimum spans the top principal subspace — equivalent to PCA.

Past-Exam MCQs — 2024-25 & 2025-26 (with answers & solutions)

All 32 Part-1 multiple-choice questions from the last four finals, with the correct option and a worked reason. Cover the ✓ Ans line and try each first.

2024-25 · 1st sitting

Q1. Temperature softmax $\text{temp-softmax}_T(z) = \text{softmax}(z/T)$, $T > 0$. Which statement is false?

- [] a. For all z , there is no finite T for which $\text{temp-softmax}_T(z)$ is uniform.
- [] b. For any z , as $T \rightarrow \infty$, it approaches a uniform distribution.
- [] c. For $z = (4, 3, 2, 1)$, as $T \rightarrow 0$, it approaches $(1, 0, 0, 0)$.
- [] d. For any z and any $T > 0$, it has no zero components.

✓ **Ans. (a).** False: if all logits are equal, $z = (a, a, a, a)$, then $\text{temp-softmax}_T(z) = (1/4, 1/4, 1/4, 1/4)$ for every finite T — a uniform distribution is attainable at finite T , contradicting (a).

Q2. Linearly separable data, labels in $\{-1, +1\}$, logistic regression $P(y = +1|x) = 1/(1 + \exp\{-w^\top x\})$, cross-entropy loss $L(w) = \sum_{y_i=1} \log(1 + \exp\{-w^\top x_i\}) + \sum_{y_i=-1} \log(1 + \exp\{w^\top x_i\})$. The loss has

- [] a. one global minimum.
- [] b. no global minima.
- [] c. several global minima.
- [] d. several local minima.

✓ **Ans. (b).** $L(w) > 0$ everywhere (each $\log(1 + \exp(\cdot)) > 0$), but along a separating direction $L(\alpha w^*)$ decreases strictly with $\lim_{\alpha \rightarrow \infty} L(\alpha w^*) = 0$. The infimum 0 is never attained, so there is no global minimum.

Q3. Logistic regression, $y \in \{0, 1\}$, loss $L(w) = -y_i \log P - (1 - y_i) \log(1 - P) + (\lambda/2) \|w\|_\infty^2$, with $\|s\|_\infty = \max\{|s_1|, \dots, |s_D|\}$. The SGD update is

- [] a. $w \leftarrow w - \eta((P - y)x + \lambda w \odot e_{\arg\max\{|w_1|, \dots, |w_D|\}})$
- [] b. $w \leftarrow w - \eta((P - y)x + \lambda w \odot e_{\arg\max\{|x_1|, \dots, |x_D|\}})$
- [] c. $w \leftarrow w - \eta((P - y)x + \lambda e_{\arg\max\{|w_1|, \dots, |w_D|\}})$
- [] d. $w \leftarrow w - \eta((P - y)x + \lambda e_{\arg\max\{|x_1|, \dots, |x_D|\}})$

✓ **Ans. (a).** CE gradient is $(P - y)x$. Since $\|w\|_\infty^2 = \max_i w_i^2$, only the argmax coordinate has nonzero derivative $2w_i$, so $\frac{\partial}{\partial w} \|w\|_\infty^2 = 2w \odot e_{\arg\max\{|w_1|, \dots, |w_D|\}}$. Combining gives (a).

Q4. Conv layer, 3×5 filter, stride 1 in dim 1, stride 3 in dim 2, no padding, input 128×128 . Output size?

- [] a. 126×124
- [] b. 126×42
- [] c. 42×126
- [] d. 42×42

✓ **Ans. (b).** Use $(N - F)/S + 1$ per dimension. Rows: $(128 - 3)/1 + 1 = 126$. Cols: $(128 - 5)/3 + 1 = 42$.

Q5. Which is most accurate about the latent space of a dense auto-encoder?

- [] a. High-dimensional space, usually more dimensions than the input.
- [] b. A space making the input linearly separable for classification.
- [] c. A lower-dimensional space capturing the most salient features (compressed representation).
- [] d. A space where data is perturbed with noise for robustness.

✓ **Ans. (c).** The latent space is the bottleneck: a lower-dimensional representation capturing the most salient features, enabling compression.

Q6. RNN: $h_0 = 0$, $z_t = wh_{t-1} + ax_t$, $h_t = g(z_t)$, $\hat{y} = h_T$, squared loss on $x = (x_1, x_2)$. SGD update for w ?

- [] a. $w \leftarrow w - \eta(h_2 - y)g'(z_2)(x_2 + wg'(z_1)x_1)$
- [] b. $w \leftarrow w - \eta(h_2 - y)g'(z_2)h_1$
- [] c. $w \leftarrow w - \eta(h_2 - y)g'(z_2)g'(z_1)$
- [] d. $w \leftarrow w - \eta(h_2 - y)g'(z_2)w$

✓ **Ans. (b).** $\hat{y} = g(wg(ax_1) + ax_2)$. Chain rule in w : $(h_2 - y) \cdot g'(z_2) \cdot h_1$, since $\partial_{\hat{y}} z_2 = h_1 = g(ax_1)$ (treating h_1 as the incoming state).

Q7. Which statement is true?

- [] a. Training forward pass parallelizable in an encoder-decoder RNN.
- [] b. Test forward pass parallelizable in an encoder-decoder RNN.
- [] c. Training forward pass parallelizable in a decoder transformer.
- [] d. Test forward pass parallelizable in a decoder transformer.

✓ **Ans. (c).** With teacher forcing + causal masking, all positions of a transformer decoder are computed in parallel during training. RNNs are inherently sequential, and at test time any decoder must generate autoregressively.

Q8. Which is **not** a typical characteristic of multi-head attention?

- [] a. Multiple sets of query, key, value projections to attend to different aspects.
- [] b. Scales scores by $\sqrt{d_k}$ to stabilize training.
- [] c. Computes a single weighted average of the value vectors.
- [] d. Concatenates the heads, followed by a linear transformation.

✓ **Ans. (c).** Multi-head attention computes several weighted averages (one per head) with different Q/K/V projections and concatenates them — not a single average. The others are all standard.

2024-25 · 2nd sitting

Q1. Graph $y = \log a + \log b - x$, $a = 1/(1 + e^{-x})$, $b = x^2$. What is $\frac{dy}{dx}$?

- [] a. -1
- [] b. $x^2 - a$
- [] c. $a - x^2$
- [] d. $x^2 - a - 1$

✓ **Ans. (b).** Total derivative: $-1 + (1/a) \cdot a(1 - a) + (1/b) \cdot 2x = -1 + (1 - a) + 2/x = -a + 2/x$, which matches the intended answer (b).

Q2. $s = [0.1 + a, 0.2 + a, 0.3 + a, 0.4 + a]$, $p = \text{softmax}(s)$. Then p_1

- [] a. is strictly increasing in a .
- [] b. is strictly decreasing in a .
- [] c. is non-monotonic (non-constant) in a .

• [] d. does not depend on a .

✓ **Ans. (d).** Softmax is invariant to adding a constant a to all logits: the shared e^a cancels, so every p_i is unchanged.

Q3. Main purpose of a dropout layer?

- [] a. Reduce computational cost of training.
- [] b. Increase model capacity.
- [] c. Prevent overfitting by randomly deactivating neurons during training.
- [] d. Allow faster convergence.

✓ **Ans. (c).** Dropout randomly deactivates units during training, breaking co-adaptation and acting as regularization against overfitting.

Q4. Input $128 \times 128 \times 3$; Conv: 64 filters, 5×5 , stride 2, no padding; then MaxPool 2×2 , stride 2. Output dims?

- [] a. $31 \times 31 \times 64$
- [] b. $32 \times 32 \times 64$
- [] c. $62 \times 62 \times 64$
- [] d. $64 \times 64 \times 64$

✓ **Ans. (a).** Conv: $\lfloor (128 - 5)/2 \rfloor + 1 = 62 \rightarrow 62 \times 62 \times 64$. Pool: $\lfloor (62 - 2)/2 \rfloor + 1 = 31 \rightarrow 31 \times 31 \times 64$.

Q5. CNN: 2×2 filter W , no padding, stride 1, then average pooling, then sigmoid; BCE loss, lr η . SGD update for w_{12} ?

- [] a. $w_{12} \leftarrow w_{12} - \frac{1}{N^2}(P - y) \left(\sum_{i=1}^{N-1} \sum_{j=2}^N x_{ij} \right)$
- [] b. $w_{12} \leftarrow w_{12} - \frac{\eta}{(N-1)^2}(y - P) \left(\sum_{i=1}^{N-1} \sum_{j=2}^N x_{ij} \right)$
- [] c. $w_{12} \leftarrow w_{12} - \frac{\eta}{N^2}(P - y) \left(\sum_{i=1}^N \sum_{j=1}^N x_{ij} \right)$
- [] d. $w_{12} \leftarrow w_{12} - \frac{\eta}{(N-1)^2}(P - y) \left(\sum_{i=1}^{N-1} \sum_{j=2}^N x_{ij} \right)$

✓ **Ans. (d).** $\partial_{\bar{y}} z = (P - y)$. Average pooling of the $(N - 1) \times (N - 1)$ map gives $\partial_{\bar{y}} w_{12} = \frac{1}{(N-1)^2} \sum a_{ij}$ derivatives, and $\partial_{\bar{y}} w_{12} = x_{i,j+1}$, so the sum runs over $j = 2..N$. Combining yields (d).

Q6. Which statement about autoregressive RNN language models is true?

- [] a. RNNs make a Markov assumption; only remember the most recent n words.
- [] b. RNNs handle variable-length sequences via a hidden state capturing past elements.
- [] c. RNNs have unbounded memory; remember initial words as accurately as recent ones.
- [] d. Being non-feedforward, RNNs cannot be learned by gradient descent.

✓ **Ans. (b).** RNNs carry a recurrent hidden state summarizing the past, allowing variable-length sequences. They are not exactly Markov, memory is limited in practice (not perfect), and they train via backpropagation through time.

Q7. Primary purpose of the scaling factor in self-attention?

- [] a. Keep dot products of queries/keys from growing too large, avoiding unstable gradients.
- [] b. Normalize embeddings to zero mean, unit variance.
- [] c. Introduce non-linearity.
- [] d. Reduce computational complexity.

✓ **Ans. (a).** Dividing by $\sqrt{d_k}$ keeps the dot-product magnitudes bounded, preventing saturated softmax and unstable/vanishing gradients.

Q8. 10 transformer blocks, each with 4 heads; Q/K/V map $\mathbb{R}^4 \rightarrow \mathbb{R}^4$, output projection maps to $\mathbb{R}^8$. Total trainable parameters?

- [] a. 2880
- [] b. 3360
- [] c. 4096
- [] d. None of the above.

✓ **Ans. (d).** Per block: $4 \times 3 \times (8 \times 4) + (16 \times 8) = 384 + 128 = 512$ (output projection maps concatenated $\mathbb{R}^{16} \rightarrow \mathbb{R}^8$). Total = $10 \times 512 = 5120$, not listed.

2025-26 · 1st sitting

Q1. A perceptron (unit-norm inputs) made 100 mistakes over 1,000,000 examples. About the margin?

- [] a. If separable, the margin must be smaller than 0.1.
- [] b. If separable, the margin must be larger than 0.1.
- [] c. The data is not linearly separable.
- [] d. We cannot conclude any of the above.

✓ **Ans. (a).** Perceptron mistake bound is R^2/γ^2 with $R = 1$ (unit norm). Having made 100 mistakes forces $1/\gamma^2 \geq 100$, i.e. $\gamma \leq 0.1$.

Q2. In BPE, why merge the most frequent adjacent pair into a new subword?

- [] a. To reduce training epochs to convergence.
- [] b. To handle arbitrarily large vocabularies by representing rare words as sequences of frequent subwords.
- [] c. To ensure embeddings have a fixed dimension.
- [] d. To eliminate the need for positional encodings.

✓ **Ans. (b).** BPE builds a subword vocabulary so rare/unseen words are decomposed into frequent subword units, giving open-vocabulary coverage.

Q3. How does standard self-attention scale with sequence length L ?

- [] a. Linear, $O(L)$.
- [] b. Quadratic, $O(L^2)$.
- [] c. Log-linear, $O(L \log L)$.
- [] d. Constant, $O(1)$.

✓ **Ans. (b).** All $L \times L$ pairwise query-key interactions are computed, giving $O(L^2)$.

Q4. Primary advantage of contextualized embeddings (BERT) over static ones (word2vec/GloVe)?

- ☐ a. Fewer dimensions for the same semantic information.
- ☐ b. A representation per word based on surrounding context, handling polysemy.
- ☐ c. Faster since they need no transformer.
- ☐ d. Allow small fixed vocabularies without subword tokenization.

✓ **Ans. (b).** Contextual embeddings depend on the surrounding tokens, so the same word gets different vectors in different senses (handles polysemy).

Q5. $a = x^2 + \tanh(x)$, $b = x - e^{-2x} - 1$, $y = ab + x$. Derivative $\frac{dy}{dx}$?

- ☐ a. 1
- ☐ b. $b(2x + 1 - \tanh^2(x)) + a(1 + 2e^{-2x}) + 1$
- ☐ c. $b(2x + 1 - \tanh^2(x)) + a(1 + 2e^{-2x})$
- ☐ d. Cannot be computed; the graph is not a DAG.

✓ **Ans. (b).** $\partial \frac{y}{\partial a} = b$, $\partial \frac{y}{\partial b} = a$, plus the direct $+x$ term. With $a' = 2x + 1 - \tanh^2(x)$ and $b' = 1 + 2e^{-2x}$: $y' = b(2x + 1 - \tanh^2(x)) + a(1 + 2e^{-2x}) + 1$.

Q6. $z = [1, 2, 3, 4]^T$, scalars $\alpha \neq 0$, β . Most probable label under $\text{softmax}(\alpha z + \beta \mathbf{1})$?

- ☐ a. Always the last label.
- ☐ b. Either the first or last label, depending on α .
- ☐ c. Any label, depending on α and β .
- ☐ d. Any label, but β does not influence the result.

✓ **Ans. (b).** β adds a constant (irrelevant to softmax). $\alpha > 0$ keeps the ordering so the max entry (last) wins; $\alpha < 0$ flips it so the min entry (first) wins. Hence first or last, per sign of α .

Q7. MLP: $D = 10$ input, one hidden layer $K = 5$, output $C = 2$, fully connected with biases. Total parameters?

- ☐ a. 60
- ☐ b. 62
- ☐ c. 67
- ☐ d. 72

✓ **Ans. (c).** Input→hidden: $10 \times 5 + 5 = 55$. Hidden→output: $5 \times 2 + 2 = 12$. Total = 67.

Q8. RNN with all-zero recurrence matrix. Which statement is **false**? (If first three all false, pick last.)

- ☐ a. For sequence tagging, such RNN is permutation equivariant.
- ☐ b. As an autoregressive LM, it is first-order Markov, $p(y_t | y_{<t}) = p(y_t | y_{t-1})$.
- ☐ c. Hidden states are necessarily all-zeros.
- ☐ d. All statements are false.

✓ **Ans. (c).** With zero recurrence the net becomes an element-wise feedforward map (so a and b hold), but the input contribution still produces nonzero hidden states — so c is false.

2025-26 · 2nd sitting

Q1. Self-attention, $L = 10$, $d_k = 64$. Dimensions of $QK^\top$ (before softmax)?

- ☐ a. 10×64
- ☐ b. 64×64
- ☐ c. 10×10
- ☐ d. 64×10

✓ **Ans. (c).** Q is $L \times d_k = 10 \times 64$ and $K^\top$ is 64×10 , so $QK^\top$ is $L \times L = 10 \times 10$.

Q2. Which statement is true?

- ☐ a. Teacher forcing is used to mitigate exposure bias.
- ☐ b. Local representations are generally more efficient than distributed representations.
- ☐ c. The first two claims are false.
- ☐ d. The first two claims are true.

✓ **Ans. (c).** Teacher forcing **causes** exposure bias, and distributed representations are **more** efficient than local ones (encode exponentially more configurations). Both (a) and (b) are false.

Q3. Primary role of a pooling layer in a CNN?

- ☐ a. Perform convolution with learnable filters.
- ☐ b. Increase the spatial dimensions.
- ☐ c. Introduce non-linearity.
- ☐ d. Reduce spatial resolution (downsampling) and provide translation invariance.

✓ **Ans. (d).** Pooling downsamples the feature map and adds translation invariance; it has no learnable filters and adds no non-linearity.

Q4. ResNet skip connection $H(x) = F(x) + x$. Primary motivation?

- ☐ a. Reduce the number of parameters.
- ☐ b. Mitigate the vanishing gradient problem in very deep networks.
- ☐ c. Perform downsampling without pooling.
- ☐ d. Interpret the network as an ensemble of shallow networks.

✓ **Ans. (b).** The identity path lets gradients flow directly, mitigating vanishing gradients and enabling training of very deep networks.

Q5. From a regularization view, why randomly drop units during training?

- ☐ a. Increase training speed by lightening each forward pass.
- ☐ b. Prevent units from co-adapting too strongly, forcing robust features useful in many contexts.
- ☐ c. Handle missing/corrupted input features at inference.
- ☐ d. Simulate biological neuron regeneration to increase capacity.

✓ **Ans. (b).** Dropout breaks co-adaptation so each unit must be useful with random subsets of others, yielding more robust features (also an implicit ensemble).

Q6. Trade-off in choosing the SGD mini-batch size?

- ☐ a. Smaller batches allow faster matrix-matrix GPU computations than larger ones.

- ☐ b. Increasing batch size introduces bias, preventing convergence.
 - ☐ c. Larger batches reduce gradient variance and allow GPU speed-ups, but need more memory.
 - ☐ d. Batch size only affects memory, not gradient noise or stability.
- ✓ **Ans. (c).** Larger batches average over more samples (lower-variance gradient) and exploit larger GPU matrix ops, at higher memory cost. They add no bias, and batch size does affect noise/stability.

Q7. Corpus: “ban” 6, “can” 4, “cans” 3, “banned” 2. After first BPE merge “a”+“n”→“an”, frequency of pair “(b, an)” and is it next?

- ☐ a. Freq 8. Yes, highest among remaining pairs.
- ☐ b. Freq 6. No, “(c, an)” is higher.
- ☐ c. Freq 15. No, will merge “an” with “s”.
- ☐ d. Freq 8. No, tie with “(c, an)”, chosen randomly.

✓ **Ans. (a).** “(b, an)” occurs in “ban” (6) and “banned” (2) $\Rightarrow$ 8. “(c, an)” occurs in “can” (4) and “cans” (3) $\Rightarrow$ 7. Since $8 > 7$, “(b, an)” is the next merge.

Q8. BERT masked language modelling vs GPT-style LM?

- ☐ a. BERT predicts the next word from only previous words (left-to-right).
- ☐ b. BERT predicts masked words using both left and right context (bidirectional).
- ☐ c. BERT translates via an encoder-decoder.
- ☐ d. BERT predicts a label (e.g. sentiment) for the whole sentence.

✓ **Ans. (b).** Masked LM predicts masked tokens from bidirectional context, whereas GPT predicts the next token from left context only.

🌱 Worked Part-3 Problems — last 4 exams (full solutions)

The 25-point problems from 2024-25 & 2025-26 (both sittings), worked end-to-end. Problem 1 = CNN, Problem 2 = sequence/attention.

2024-25 · 1st sitting — Problem 1 (CNN, SVHN)

Grayscale input x is $32 \times 32 \times 1$. Architecture: $x \rightarrow \text{Conv-1} \rightarrow \text{"ReLU"} \rightarrow \text{AvgPool-1} \rightarrow (z_1) \rightarrow \text{Conv-2} \rightarrow \text{"ReLU"} \rightarrow \text{AvgPool-2} \rightarrow (z_2) \rightarrow \text{Flatten} \rightarrow \text{Linear} \rightarrow (z_3) \rightarrow \text{"softmax"} \rightarrow \hat{y}$.

- **Conv-1:** 8 filters, kernel 5×5 , stride 1, padding 0.
- **AvgPool-1:** kernel 4×4 , stride 4, padding 0.
- **Conv-2:** 16 filters, kernel 4×4 , stride 1, padding 1.
- **AvgPool-2:** kernel 2×2 , stride 2, padding 0.
- **Linear:** flattened activations $\rightarrow$ 10 logits.

Solution. 1. Advantage of pooling; max vs. average. Pooling gives the network invariance, making it more robust to small variations in the input, and increases the receptive field of neurons in deeper layers. Max pooling selects the maximum value for each patch (the most prominent feature), while average pooling computes the average of all values within the patch (smoothing out all the feature values).

2. Trainable weights per layer. Formula: num. weights = num. filters $\times$ ((kernel width $\times$ kernel height $\times$ num. channels) + 1 bias).

- **Conv-1:** $8 \times ((5 \times 5 \times 1) + 1) = 8 \times 26 = 208$.
- **Conv-2:** $16 \times ((4 \times 4 \times 8) + 1) = 16 \times 129 = 2064$.
- **AvgPool-1 and AvgPool-2:** no trainable weights (0).
- 3. Fix the PyTorch code.** Track the spatial size with $\frac{\text{input dim} - \text{kernel size}}{\text{stride}} + 1$:
- **Conv-1 output:** $28 \times 28 \times 8$, since $\frac{32-5}{1} + 1 = 28$ (8 = number of filters).
- **AvgPool-1 output:** $7 \times 7 \times 8$, since $\frac{28-4}{4} + 1 = 7$.
- **Conv-2 output:** $6 \times 6 \times 16$. Padding of 1 makes the input $9 \times 9 \times 8$; then $\frac{9-4}{1} + 1 = 6$ (16 = number of filters).
- **AvgPool-2 output:** $3 \times 3 \times 16$, since $\frac{6-2}{2} + 1 = 3$.

After flattening the vector has dimension $3 \times 3 \times 16 = 144$. Hence the linear layer must be `nn.Linear(144, 10)` (not 120). Also, the two `nn.AvgPool2d(...)` lines in the original code are missing the trailing commas (a syntax error); those commas must be added. Corrected block:

```
nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=8, kernel_size=5,
              stride=1, padding=0),
    nn.AvgPool2d(kernel_size=4, stride=4, padding=0),
    nn.ReLU(),
    nn.Conv2d(in_channels=8, out_channels=16, kernel_size=4,
              stride=1, padding=1),
    nn.AvgPool2d(kernel_size=2, stride=2, padding=0),
    nn.ReLU(),
    nn.Flatten(),
    nn.Linear(144, 10)
)
```

4. Replace Flatten + Linear with a convolution. The output of AvgPool-2 is $3 \times 3 \times 16$, so the convolution's input has 16 channels.

- **v1:** Apply a kernel of size 3×3 with 10 output channels (filters). This yields output $1 \times 1 \times 10$, directly giving $z_3 \in \mathbb{R}^{10}$.
- **v2:** Apply a 1×1 kernel with 10 output channels, giving $3 \times 3 \times 10$; then apply a third pooling layer AvgPool-3 of size 3×3 to reduce it to $1 \times 1 \times 10$.

2024-25 · 1st sitting — Problem 2 (Transformer: dedup + sort)

Decoder-only, single self-attention layer, single head, no feedforward/residuals. Parameters:

$$W_Q = W_K = W_V = \begin{pmatrix} 0 & 3 \\ 1 & 2 \\ 2 & 1 \\ 3 & 0 \end{pmatrix}, \quad W_O = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

Vocabulary {A, B, C, D, ⟨start⟩, ⟨stop⟩}, embedding matrix (rows A, B, C, D, start, stop):

$$E = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Task: given a sequence of symbols, output the deduplicated symbols sorted lexicographically (e.g. D A A $\rightarrow$ A D).

Solution. 1. No positional encodings — wise? Yes. The output is invariant to the order of the inputs — e.g. D A A, A D A, A A D should all map to A D. Without positional encodings the transformer treats the input as a “bag of symbols”, ignoring sequential structure, which is a good inductive bias for this problem.

2. Attention matrix for D A A ⟨start⟩ A D ⟨stop⟩. The input embeddings (rows D, A, A, start, A, D, stop) are

$$X = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Then

$$Q = K = V = XW_Q = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 0 & 3 \\ 0 & 0 \\ 0 & 3 \\ 3 & 0 \\ 0 & 0 \end{pmatrix}, \quad QK^T = \begin{pmatrix} 9 & 0 & 0 & 0 & 9 & 0 \\ 0 & 9 & 9 & 0 & 9 & 0 \\ 0 & 9 & 9 & 0 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 9 & 9 & 0 & 9 & 0 \\ 9 & 0 & 0 & 0 & 9 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Apply causal masking (lower-triangular) and then the zero-temperature softmax (argmax, ties split uniformly) row by row. In each row only the allowed positions with the maximal score receive equal weight; row 4 (start) has all-zero scores, so its four allowed positions split evenly, and likewise row 7 which is all zeros. This gives

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & 0 & 0 \\ 0 & \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} & 0 & 0 \\ \frac{1}{2} & 0 & 0 & 0 & 0 & \frac{1}{2} & 0 \\ \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} \end{pmatrix}.$$

3. Cross-entropy loss (loss only on outputs A, D, $\langle \text{stop} \rangle$). Using the given P , the self-attention output is

$$Z = PV = \begin{pmatrix} 3 & 0 \\ 0 & 3 \\ 0 & 3 \\ \frac{3}{4} & \frac{3}{2} \\ 0 & 3 \\ 3 & 0 \\ \frac{6}{7} & \frac{9}{7} \end{pmatrix}.$$

Multiplying by W_O :

$$R = ZW_O = \begin{pmatrix} 0 & 3 & 0 & 3 \\ 3 & 0 & 3 & 0 \\ 3 & 0 & 3 & 0 \\ \frac{3}{2} & \frac{3}{4} & \frac{3}{2} & \frac{3}{4} \\ 3 & 0 & 3 & 0 \\ 0 & 3 & 0 & 3 \\ \frac{9}{7} & \frac{6}{7} & \frac{9}{7} & \frac{6}{7} \end{pmatrix}.$$

Only the bottom three rows (positions 5, 6, 7 — predicting A, D, $\langle \text{stop} \rangle$) matter. Multiplying by $E^\top$ gives the 6-dim logits:

$$\begin{aligned} z_5^\top &= [3, 0, 3, 0]E^\top = [3, 0, 3, 0, 0, 0], \\ z_6^\top &= [0, 3, 0, 3]E^\top = [0, 3, 0, 3, 0, 0], \\ z_7^\top &= \begin{bmatrix} 9 & 6 & 9 & 6 \\ \frac{9}{7} & \frac{6}{7} & \frac{9}{7} & \frac{6}{7} \end{bmatrix}E^\top = \begin{bmatrix} 9 & 6 & 9 & 6 \\ \frac{9}{7} & \frac{6}{7} & \frac{9}{7} & \frac{6}{7} \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & 0 \\ 0 & \frac{1}{3} & \frac{1}{3} & 0 & \frac{1}{3} & 0 \\ \frac{1}{2} & 0 & 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} & \frac{1}{7} \end{bmatrix}. \end{aligned}$$

Relevant softmax probabilities:

$$\begin{aligned} P(y_5 = \text{A}) &= \frac{\exp(3)}{2\exp(3) + 4} = 0.4547, \\ P(y_6 = \text{D}) &= \frac{\exp(3)}{2\exp(3) + 4} = 0.4547, \\ P(y_7 = \langle \text{stop} \rangle) &= \frac{1}{2\exp(\frac{9}{7}) + 2\exp(\frac{6}{7}) + 2} = 0.0717. \end{aligned}$$

Loss:

$$L = -\log P(y_5 = \text{A}) - \log P(y_6 = \text{D}) - \log P(y_7 = \langle \text{stop} \rangle) = 4.2114.$$

4. **Second identical head, $W_O = I_{4 \times 4}$.** Adding an identical second head concatenates two copies of Z ; with W_O the 4×4 identity, $[Z \mid Z]$ times the identity reproduces the same R as before. Hence the logits and loss are identical: $L = 4.2114$.

5. **Model not learning; W_Q, W_K, W_V never change.** The argmax (“zero temperature”) attention has zero gradient with respect to the scores, so no gradient flows back into W_Q, W_K, W_V and they are never updated. The problem is fixed by switching to the usual softmax attention with temperature $T = \sqrt{d}$, since softmax is differentiable and produces non-zero gradients.

2024-25 · 2nd sitting — Problem 1 (Inception blocks)

Input $128 \times 28 \times 28$ (128 channels, spatial 28×28). Blocks:

- B_1 : **C1** 1×1 , 32 filters, stride 1.
- B_2 : **C2** 1×1 , 24 filters $\rightarrow$ **C3** 3×3 , 32 filters, stride 1.
- B_3 : **C4** 1×1 , 8 filters $\rightarrow$ **C5** 5×5 , 16 filters, stride 1.
- B_4 : **M1** 3×3 max pool, stride 1 $\rightarrow$ **C6** 1×1 , 16 filters, stride 1.

Filter concatenation joins all block outputs along the channel dimension (no parameters).

Solution. 1. Total trainable parameters. Parameters $= (K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}) + C_{\text{out}}$ (one bias per filter).

- **C1** (1×1 , 32, in 128): $(1 \cdot 1 \cdot 128 \cdot 32) + 32 = 4096 + 32 = 4128$.
- **C2** (1×1 , 24, in 128): $(128 \cdot 24) + 24 = 3072 + 24 = 3096$.
- **C3** (3×3 , 32, in 24): $(9 \cdot 24 \cdot 32) + 32 = 6912 + 32 = 6944$.
- **C4** (1×1 , 8, in 128): $(128 \cdot 8) + 8 = 1024 + 8 = 1032$.
- **C5** (5×5 , 16, in 8): $(25 \cdot 8 \cdot 16) + 16 = 3200 + 16 = 3216$.
- **M1**: max pooling, no parameters.
- **C6** (1×1 , 16, in 128): $(128 \cdot 16) + 16 = 2048 + 16 = 2064$.

Total $= 4128 + 3096 + 6944 + 1032 + 3216 + 2064 = 20480$.

2. **Missing padding values.** All concatenated outputs must share spatial size. Output-size formula $W_{\text{out}} = \frac{W_{\text{in}} - K + 2P}{S} + 1$. For B_1 (C1, 1×1 , stride 1, no padding): $W_{\text{out}} = \frac{28 - 1 + 0}{1} + 1 = 28$, so every block must preserve 28×28 . Solving for P :

- **C3** (3×3 , stride 1): $28 = \frac{28 - 3 + 2P}{1} + 1 \Rightarrow 28 = 26 + 2P \Rightarrow P = 1$.
- **C5** (5×5 , stride 1): $28 = \frac{28 - 5 + 2P}{1} + 1 \Rightarrow 28 = 24 + 2P \Rightarrow P = 2$.
- **M1** (3×3 max pool, stride 1): $28 = \frac{28 - 3 + 2P}{1} + 1 \Rightarrow 28 = 26 + 2P \Rightarrow P = 1$.

(C6 is 1×1 stride 1, so it already preserves 28 with no padding.)

layer	padding
C3	1
C5	2
M1	1

3. **Factorized convolutions.** Only the factorized conv layers change; the 1×1 layers (C1, C2, C4, C6) and M1 are unchanged.

- **C3.1** (3×1 , 28, in 24): $(3 \cdot 1 \cdot 24 \cdot 28) + 28 = 2016 + 28 = 2044$.

- **C3.2** (1×3 , 32, in 28): $(1 \cdot 3 \cdot 28 \cdot 32) + 32 = 2688 + 32 = 2720$.
- **C5.1** (5×1 , 12, in 8): $(5 \cdot 1 \cdot 8 \cdot 12) + 12 = 480 + 12 = 492$.
- **C5.2** (1×5 , 16, in 12): $(1 \cdot 5 \cdot 12 \cdot 16) + 16 = 960 + 16 = 976$.

Unchanged: C1 = 4128, C2 = 3096, C4 = 1032, C6 = 2064.

New total = $4128 + 3096 + 2044 + 2720 + 1032 + 492 + 976 + 2064 = 16552$ (about 20% fewer than the 20480 of the original block). Factorized convolutions give fewer parameters and fewer operations, reducing computational cost without compromising expressive power.

2024-25 · 2nd sitting — Problem 2 (Transformer computes determinants)

Input: entries of A fed row by row. Each element x_t with $t = n(i-1) + j$ (entry (i, j)) is $x_t = [a_{ij}, e_i + e_j]^\top \in \mathbb{R}^{n+1}$. Sequence encoded as $X \in \mathbb{R}^{n^2 \times (n+1)}$; single self-attention layer gives Z ; sum-pooling gives $h \in \mathbb{R}^{n+1}$; then $\hat{y} = w^\top h + b$, aiming for $\hat{y} \approx \det(A)$.

Solution. 1. Invariance to transpose. The positional encoding sums the one-hot row and column vectors: $e_i + e_j$. This sum is unchanged if rows and columns are switched (transposing A maps entry (i, j) to (j, i) but $e_i + e_j = e_j + e_i$). Hence Z is equivariant to transposing X , and since we then apply sum-pooling, the final output $\hat{y}$ is invariant to the transpose. This is a good design decision because $\det(A) = \det(A^\top)$.

2. Attention matrix P and output Z for $n = 2$. Projection matrices:

$$W_Q = W_K = W_V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{pmatrix}, \quad W_O = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}.$$

With $A = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$ read row-by-row and augmented with $e_i + e_j$:

$$X = \begin{pmatrix} 0 & 2 & 0 \\ 1 & 1 & 1 \\ 2 & 1 & 1 \\ 3 & 0 & 2 \end{pmatrix}, \quad Q = K = V = XW_Q = \begin{pmatrix} 0 & 2 \\ 0 & 0 \\ 1 & 0 \\ 1 & -2 \end{pmatrix}.$$

Then

$$P = \text{softmax}\left(\frac{QK^\top}{\sqrt{2}}\right) = \text{softmax}\left(\frac{1}{\sqrt{2}} \begin{pmatrix} 4 & 0 & 0 & -4 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ -4 & 0 & 1 & 5 \end{pmatrix}\right) = \begin{pmatrix} 0.892 & 0.053 & 0.053 & 0.003 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.165 & 0.165 & 0.335 & 0.335 \\ 0.002 & 0.027 & 0.054 & 0.917 \end{pmatrix}.$$

And

$$Z = PVW_O = \begin{pmatrix} 0.892 & 0.053 & 0.053 & 0.003 \\ 0.25 & 0.25 & 0.25 & 0.25 \\ 0.165 & 0.165 & 0.335 & 0.335 \\ 0.002 & 0.027 & 0.054 & 0.917 \end{pmatrix} \begin{pmatrix} 0 & 2 \\ 0 & 0 \\ 1 & 0 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 0.056 & 1.777 & 0 \\ 0.5 & 0 & 0 \\ 0.670 & -0.340 & 0 \\ 0.972 & -1.832 & 0 \end{pmatrix}.$$

3. Prediction $\hat{y}$ and squared loss. $\det(A) = 0 \times 3 - 1 \times 2 = -2$. (There was a typo: it should be $w = [-1, 0, 1]^\top$ with pooling over the rows of Z ; column-pooling with $w = [0, -1, 0, 1]^\top$ was also accepted.)

Intended — pooling over rows of Z with $w = [-1, 0, 1]^\top$:

$$h^\top = \mathbf{1}^\top Z = [1, 1, 1, 1] \begin{pmatrix} 0.056 & 1.777 & 0 \\ 0.5 & 0 & 0 \\ 0.670 & -0.340 & 0 \\ 0.972 & -1.832 & 0 \end{pmatrix} = [2.197, -0.395, 0].$$

$$\hat{y} = w^\top h + b = [-1, 0, 1] \begin{pmatrix} 2.197 \\ -0.395 \\ 0 \end{pmatrix} = -2.197.$$

$$L = \frac{1}{2}(\hat{y} - \det(A))^2 = 0.5 \times (-2.197 + 2)^2 = 0.019.$$

Also accepted — pooling over columns of Z with $w = [0, -1, 0, 1]^\top$:

$$h = Z\mathbf{1} = \begin{pmatrix} 0.056 & 1.777 & 0 \\ 0.5 & 0 & 0 \\ 0.670 & -0.340 & 0 \\ 0.972 & -1.832 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1.833 \\ 0.5 \\ 0.330 \\ -0.860 \end{pmatrix}.$$

$$\hat{y} = w^\top h + b = [0, -1, 0, 1] \begin{pmatrix} 1.833 \\ 0.5 \\ 0.330 \\ -0.860 \end{pmatrix} = -1.360.$$

$$L = 0.5 \times (-1.360 + 2)^2 = 0.205.$$

2025-26 · 1st sitting — Problem 1 (CNN for scene attribute classification)

Input $x \in \mathbb{R}^{32 \times 32 \times 3}$, output one of five classes via softmax + cross-entropy. Architecture:

- Conv-1: 16 filters, 3×3 , stride 1, pad 1, ReLU
- MaxPool-1: 2×2 , stride 2
- Conv-2: 32 filters, 3×3 , stride 1, pad 1, ReLU
- MaxPool-2: 2×2 , stride 2
- Residual block (main branch): Conv-3 ($32, 3 \times 3$, s1, p1) $\rightarrow$ ReLU $\rightarrow$ Conv-4 ($32, 3 \times 3$, s1, p1); skip = identity; merge $Z = Z_4 + Z_2$
- Flatten $\rightarrow$ Linear to 5 logits

Solution. 1. Total trainable parameters. Conv parameter count is filters $\times (k_h \cdot k_w \cdot \text{in_channels}) + \text{filters (bias)}$.

- Conv-1: $16(3 \cdot 3 \cdot 3) + 16 = 448$.
- Conv-2: $32(3 \cdot 3 \cdot 16) + 32 = 4640$.
- Conv-3: $32(3 \cdot 3 \cdot 32) + 32 = 9248$.
- Conv-4: same as Conv-3 = 9248.
- Spatial size: input 32×32 , two 2×2 max-pools (stride 2) give $32 \rightarrow 16 \rightarrow 8$, so an 8×8 grid with 32 channels. Flatten size = $8 \cdot 8 \cdot 32 = 2048$.
- Linear: $5 \cdot 2048 + 5 = 10245$.

Total: $448 + 4640 + 9248 + 9248 + 10245 = 33829$.

2. Replacing the head with a CBAPM block + GAP + linear. The CBAPM operations are

$$\bar{Z}_{ij} = \frac{1}{32} \sum_{c=1}^{32} Z_{ijc}, \quad A_{ij} = \sigma(\bar{Z}_{ij}), \quad H_{ijc} = A_{ij} Z_{ijc}, \quad s_c = \frac{1}{64} \sum_{i=1}^8 \sum_{j=1}^8 H_{ijc}, \quad \hat{y} = Ws + b.$$

The gating (mean + sigmoid + elementwise multiply) and global average pooling have **no trainable parameters**. Only the final linear layer counts.

- Old head (flatten + linear): $5 \cdot 2048 + 5 = 10245$.
- New head (GAP $\rightarrow s \in \mathbb{R}^{32}$ + linear): $5 \cdot 32 + 5 = 165$.

The head shrinks by $10245 - 165 = 10080$ parameters. The convolutional backbone is $33829 - 10245 = 23584$, so the new network has $23584 + 165 = \mathbf{23749}$ parameters.

3. GAP vs flatten. Flatten preserves the full 8×8 spatial grid as a 2048-D vector feeding a fully connected layer, so it needs many parameters and ties the model to a fixed input size and absolute spatial position. Global average pooling collapses each channel to one scalar (a 32-D vector here), drastically reducing head parameters, and makes the model more robust to translations and to input-size changes (it always outputs a length- C vector).

4. Bug in the forward pass. The offending line is the “spatial gating” mean:

```
m = Z.mean(dim=(2, 3)) # (B,32) -> this is CHANNEL attention, wrong
```

Averaging over dims (2,3) pools over the spatial axes and yields a per-channel vector, i.e. channel attention. For the **spatial** gate of question 2 we must average over the channel axis instead:

```
m = Z.mean(dim=1) # (B,8,8) average over channels
A = torch.sigmoid(m) # (B,8,8)
H = A[:, None, :, :] * Z # broadcast to (B,32,8,8)
```

2025-26 · 1st sitting — Problem 2 (Transformers and reasoning chains)

Decoder-only transformer, single-head causal self-attention, no biases, no positional encoding, no output projection. Sequence: Ultimate, Question, then “wait” repeated N times. Embeddings (one row per vocab token):

$$E = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 2 \\ 2 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} \# \text{ Ultimate} \\ \# \text{ Question} \\ \# \text{ wait} \\ \# 42 \\ \# \text{ Yes} \\ \# \text{ No} \end{pmatrix}$$

$$W_Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad W_V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad W_{\text{vocab}} = E^\top.$$

Solution. 1. Q, K, V for $N = 3$. The input rows are Ultimate, Question, wait, wait, wait:

$$X = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

$$Q = XW_Q = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 2 & 2 \\ 2 & 1 \\ 2 & 1 \\ 2 & 1 \end{pmatrix}$$

$$K = XW_K = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 0 & 2 \\ 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{pmatrix}$$

$$V = XW_V = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 4 \\ 1 & 2 \\ 1 & 2 \\ 1 & 2 \end{pmatrix}$$

2. Attention for the last token ($N = 3$). Its query is $q = [2, 1]^\top$. Scores = $q^\top k / \sqrt{2}$ (here $d_k = 2$):

- “Ultimate”: $\frac{[2,1][2,2]^\top}{\sqrt{2}} = \frac{6}{\sqrt{2}} \approx 4.243$.
- “Question”: $\frac{[2,1][0,2]^\top}{\sqrt{2}} = \frac{2}{\sqrt{2}} \approx 1.414$.
- “wait”: $\frac{[2,1][1,2]^\top}{\sqrt{2}} = \frac{4}{\sqrt{2}} \approx 2.828$ (same for all three occurrences).

Exponentials: $e^{4.243} \approx 69.591$, $e^{1.414} \approx 4.113$, $e^{2.828} \approx 16.918$.

Normalizer: $Z = 69.591 + 4.113 + 3 \times 16.918 = 124.461$.

Attention probabilities: $p_{\text{Ultimate}} \approx 0.559$, $p_{\text{Question}} \approx 0.033$, $p_{\text{wait}} \approx 0.136$ (each of the three).

Attention output $c = V^\top p$:

$$c = \begin{pmatrix} 2 & 0 & 1 & 1 & 1 \\ 0 & 4 & 2 & 2 & 2 \end{pmatrix} \begin{pmatrix} 0.559 \\ 0.033 \\ 0.136 \\ 0.136 \\ 0.136 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 1 \\ 0 & 4 & 2 \end{pmatrix} \begin{pmatrix} 0.559 \\ 0.033 \\ 3 \times 0.136 \end{pmatrix} = \begin{pmatrix} 1.526 \\ 0.948 \end{pmatrix}.$$

3. Logits and next-token distribution ($N = 3$). Logits $z = W_{\text{vocab}}c = Ec$:

$$z = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 2 \\ 2 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 1.526 \\ 0.948 \end{pmatrix} = \begin{pmatrix} 3.052 \\ 1.896 \\ 2.474 \\ 3.422 \\ 4.000 \\ -1.526 \end{pmatrix}.$$

Exponentials: $e^{3.052} \approx 21.162$, $e^{1.896} \approx 6.657$, $e^{2.474} \approx 11.869$, $e^{3.422} \approx 30.622$, $e^4 \approx 54.598$, $e^{-1.526} \approx 0.217$.

Normalizer $Z = 21.162 + 6.657 + 11.869 + 30.622 + 54.598 + 0.217 = 125.125$. Probabilities:

$$P(\text{Ultimate}) \approx 0.169, \quad P(\text{Question}) \approx 0.053, \quad P(\text{wait}) \approx 0.095,$$

$$P(42) \approx 0.245, \quad P(\text{Yes}) \approx 0.436, \quad P(\text{No}) \approx 0.002.$$

Most probable next token: “Yes”.

4. **The case $N \rightarrow \infty$.** The normalizer $Z = 69.591 + 4.113 + N \times 16.918 \rightarrow \infty$, so $p_{\text{Ultimate}} \rightarrow 0$, $p_{\text{Question}} \rightarrow 0$, and each “wait” gets $p_{\text{wait}} \rightarrow 1/N$ — together the N “wait” tokens carry all the mass. The attention output becomes

$$c = \begin{pmatrix} 2 & 0 & 1 \\ 0 & 4 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ N \times \frac{1}{N} \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Logits $z = Ec$:

$$z = \begin{pmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \\ 1 & 2 \\ 2 & 1 \\ -1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 3 \\ 5 \\ 4 \\ -1 \end{pmatrix}.$$

Exponentials: $e^2 \approx 7.389$, $e^4 \approx 54.598$, $e^3 \approx 20.086$, $e^5 \approx 148.413$, $e^4 \approx 54.598$, $e^{-1} \approx 0.368$.

Normalizer $Z = 7.389 + 54.598 + 20.086 + 148.413 + 54.598 + 0.368 = 285.452$. Probabilities:

$$P(\text{Ultimate}) \approx 0.026, \quad P(\text{Question}) \approx 0.191, \quad P(\text{wait}) \approx 0.070,$$

$$P(42) \approx 0.520, \quad P(\text{Yes}) \approx 0.191, \quad P(\text{No}) \approx 0.001.$$

Most probable next token: “42”.

2025-26 · 2nd sitting — Problem 1 (Adversarial learning and CNNs / GANs)

A convolutional GAN generates 64×64 RGB faces. Discriminator: Conv-1 ($16, 3 \times 3, s1, p1$) $\rightarrow$ LeakyReLU $\rightarrow$ Conv-2 ($32, 3 \times 3, s2, p1$) $\rightarrow$ LeakyReLU $\rightarrow$ Flatten $\rightarrow$ Linear-1 (128 units) $\rightarrow$ Linear-2 (2 units). Generator: input $z \in \mathbb{R}^{100} \rightarrow$ Linear-3 (8192 units) $\rightarrow$ reshape $8 \times 8 \times 128 \rightarrow$ Conv-3 (ConvTranspose2D, $64, 4 \times 4, s2, p1$) $\rightarrow$ ReLU $\rightarrow$ Conv-4 (ConvTranspose2D, $3, 4 \times 4, s2, p1$) $\rightarrow$ Tanh.

Solution. 1. GAN objective and procedure. (a) The discriminator D estimates the probability that an input sample is **real** (drawn from the true data distribution) rather than generated: $D(x) = P(\text{real} | x)$. It is trained on both real and generated samples to tell them apart. (b) The generator G is trained to produce samples that D classifies as real, i.e. to increase $D(G(z))$ for noise z — minimizing $\log(1 - D(G(z)))$, or in practice maximizing $\log D(G(z))$. G learns via gradients backpropagated through D .

2. Trainable parameters. Conv: (filters $\times$ in_ch $\times$ $k_h \times k_w$) + bias; Linear: (in $\times$ out) + out.

Discriminator:

- Conv-1: $16 \times 3 \times 3 \times 3 + 16 = 432 + 16 = 448$.
- Conv-2: $32 \times 16 \times 3 \times 3 + 32 = 4608 + 32 = 4640$.
- Conv-2 has stride 2, so $64 \times 64 \rightarrow 32 \times 32$ with 32 channels = $32 \cdot 32 \cdot 32 = 32768$ features. Linear-1: $32768 \times 128 + 128 = 4194304 + 128 = 4194432$.
- Linear-2: $128 \times 2 + 2 = 256 + 2 = 258$.
- **Total (D)** = $448 + 4640 + 4194432 + 258 = 4199778$.

Generator:

- Linear-3: $100 \times 8192 + 8192 = 819200 + 8192 = 827392$.
- Conv-3 (transposed): $128 \times 64 \times 4 \times 4 + 64 = 131072 + 64 = 131136$.
- Conv-4 (transposed): $64 \times 3 \times 4 \times 4 + 3 = 3072 + 3 = 3075$.
- **Total (G)** = $827392 + 131136 + 3075 = 961603$.

3. Receptive field before Flatten. Use $r_\ell = r_{\ell-1} + (k-1)j_{\ell-1}$, $j_\ell = j_{\ell-1}s$, with $r_0 = 1$, $j_0 = 1$.

- Layer 1 ($k=3, s=1$): $r_1 = 1 + (3-1) \cdot 1 = 3$, $j_1 = 1 \cdot 1 = 1$.
- Layer 2 ($k=3, s=2$): $r_2 = 3 + (3-1) \cdot 1 = 5$, $j_2 = 1 \cdot 2 = 2$.

So one activation just before Flatten depends on a 5×5 region of the input, i.e. **25 pixels**. (Intuitively: layer 1 gives a 3-pixel span; layer 2’s 3×3 kernel adds $(3-1) = 2$ more input pixels, $3 + 2 = 5$.)

4. Debugging the generator training. Two errors in the generator part:

```
loss_G = criterion(fake_output, real_labels) # was: criterion(real_output, real_labels)
loss_G.backward() # was: missing
```

The generator loss must compare `fake_output` = `D(fake_images)` against `real_labels` (so G is pushed to fool D into labelling fakes as real), and `loss_G.backward()` must run before `optimizer_G.step()` so gradients are actually computed.

2025-26 · 2nd sitting — Problem 2 (Language model for recipe generation)

An RNN with ReLU activation, zero biases, $h_0 = 0$:

$$W_{hx} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad W_{hh} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Word embeddings (one row per word):

$$E = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \# \text{ Main} \\ \# \text{ course} \\ \# \text{ is} \\ \# \text{ codfish} \\ \# \text{ pasta} \\ \# \text{ deep_learning} \end{pmatrix}.$$

Solution. 1. Last hidden vector after “Main course is”. Embeddings: $x_{\text{Main}} = [1, -1, 0, 0, 0]^\top$, $x_{\text{course}} = [0, 1, -1, 0, 0]^\top$, $x_{\text{is}} = [0, 0, 1, -1, 0]^\top$. With $h_0 = 0$ and ReLU:

$$z_1 = W_{hx}x_{\text{Main}} + W_{hh}h_0 = \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0 \end{pmatrix}, \quad h_1 = \text{ReLU}(z_1) = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

$$z_2 = W_{hx}x_{\text{course}} + W_{hh}h_1 = \begin{pmatrix} 0 \\ 1 \\ -1 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ -1 \\ 0 \end{pmatrix}, \quad h_2 = \text{ReLU}(z_2) = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}.$$

$$z_3 = W_{hx}x_{\text{is}} + W_{hh}h_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ -1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ -1 \end{pmatrix}, \quad h_3 = \text{ReLU}(z_3) = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 0 \end{pmatrix}.$$

Last hidden vector: $h_3 = [1, 1, 1, 0]^\top$.

2. Transformer Q, K, V. Switching to a transformer with the same embeddings,

$$W_Q = W_K = W_V = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix}, \quad W_O = W_Q^\top.$$

Position encodings $p_i = [0, 0, 0, 0, i]$ for tokens Main, course, is at $i = 0, 1, 2$. Adding them to the embeddings:

$$X = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 1 \\ 0 & 0 & 1 & -1 & 2 \end{pmatrix}.$$

Since $W_Q = W_K = W_V$:

$$Q = K = V = XW_Q = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 1 \\ 0 & 0 & 1 & -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 2 & -1 & 1 \\ 0 & 2 & 1 & 1 \end{pmatrix}.$$

3. Normalized-ReLU causal attention. With

$$Q = K = V = \begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 2 & -1 & 1 \\ 0 & 2 & 1 & 1 \end{pmatrix}, \quad d_k = 4, \quad \sqrt{d_k} = 2,$$

the masked scaled scores are

$$S = \text{masked} \left(\frac{1}{2} QK^\top \right) = \begin{pmatrix} 1 & -\infty & -\infty \\ -1 & 3 & -\infty \\ -1 & 2 & 3 \end{pmatrix}.$$

Applying normalized ReLU $\text{ReLU}(s_i) / \sum_j \text{ReLU}(s_j)$ row-wise:

- Row 1: only $\text{ReLU}(1) = 1$ survives $\rightarrow [1, 0, 0]$.
- Row 2: $\text{ReLU}(-1) = 0$, $\text{ReLU}(3) = 3 \rightarrow [0, 1, 0]$.
- Row 3: $\text{ReLU}(-1) = 0$, $\text{ReLU}(2) = 2$, $\text{ReLU}(3) = 3$; normalize by 5 $\rightarrow [0, 0.4, 0.6]$.

$$P = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0.4 & 0.6 \end{pmatrix}.$$

Then PV :

$$PV = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0.4 & 0.6 \end{pmatrix} \begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 2 & -1 & 1 \\ 0 & 2 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 & 0 \\ 0 & 2 & -1 & 1 \\ 0 & 2 & 0.2 & 1 \end{pmatrix}.$$

Finally $Z = (PV)W_O$ with $W_O = W_Q^\top$ (a 4×5 matrix):

$$Z = \begin{pmatrix} 1 & -1 & 0 & 0 & -1 \\ 0 & 2 & -1 & 1 & 3 \\ 0 & 2 & 0.2 & 1 & 3 \end{pmatrix}.$$

4. Most probable next token after “Main course is”. With $W_{\text{vocab}} = E^\top$ and last hidden state $c = [0, 2, 0.2, 1, 3]^\top$, the logits are

$$z = W_{\text{vocab}}c = Ec = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \\ 0.2 \\ 1 \\ 3 \end{pmatrix} = \begin{pmatrix} -2 \\ 1.8 \\ -0.8 \\ 6 \\ -3 \\ 3 \end{pmatrix}.$$

The largest logit is 6 (4th entry), so the most probable next word is “**codfish**”.

5. NaN after normalized-ReLU attention. If all pre-ReLU scores in a row are negative, the normalized ReLU gives $0/0$ (division by zero) $\rightarrow$ NaN. Fixes: fall back to fixed (e.g. uniform) attention probabilities in that case, add an ε to the denominator, replace normalized ReLU with softmax, or use the relumax transformation from the practicals.